1. OVERVIEW

1.1 Introduction

This assignment studies brute-force and exhaustive-search algorithms on seven realistic problems: sorting e-commerce orders, finding a tool in a hangar, screening security logs, finding the two closest drones, drawing a reserve boundary, and loading a relief helicopter. For every question I followed the same cycle the assignment asks for: **predict → design → implement → measure → analyse → evaluate**. In Q7 I then **diagnosed, redesigned and validated** the solution with the worst limitation.

All seven programs were written in Python and run on my own laptop. Each program counts the operation that decides its cost — comparisons, swaps, character comparisons, pairs, orientation tests or subsets — and these counts are the primary evidence in this report. Execution time is reported as supporting evidence only.

Table 1.1: Assignment Details

Item	Details
Course	Design and Analysis of Algorithms — Assignment 2 (CO2)
Topic	Brute-force and exhaustive-search techniques
Student name	Jason Joshva M J
Registration number	192511209
Programme	B.E.CSE
Dataset seed	1209 (last four digits of the registration number)
Implementation	Python 3.13.14, run in the VS Code terminal (PowerShell) on Windows
Date of the experimental runs	16 October 2026 (21:03–21:25)
Source code and raw data	https://github.com/jasonjoshva4-max/CSA0617-DAA

1.2 How the Evidence Was Produced

Every program follows the same rules, so that results from different questions can be trusted in the same way:

- **Personal data.** Every random dataset is generated from the seed 1209, so the data, counts and graphs belong to this run only. All raw data is saved in `results/` (measurements) and `results/data/` (inputs).
- **Counting the basic operation.** A counter is increased exactly where the algorithm performs its basic operation. The counters do not change what the algorithm does.

- **Prediction first.** The predictions in this report were derived from theory before the experiments were run. Each program prints the predicted value next to the measured one, and in Q1, Q3, Q6 and Q7 it prints its predictions before running the experiment.
- **Independent checks.** Every answer is checked against a different method: Python's `sorted()`, `str.startswith`, a monotone-chain hull, a dynamic-programming knapsack, or a hand calculation.
- **Time as supporting evidence.** Times come from `time.perf_counter()`, averaged over repeated runs where a run is short. They include the counter overhead and vary between runs, so no conclusion relies on time alone.

Table 1.2: Primary Measure and Correctness Check for Each Question

Q	Problem	Primary measure	Independent check
Q1	Sorting orders by dispatch priority	Key comparisons, swaps/shifts, writes	Python sorted(); swaps = inversions
Q2	Tool lookup in a hangar	Key comparisons per lookup	Position of target known in advance
Q3	Signature search in system logs	Character comparisons	str.startswith at every position
Q4	Closest pair of drones	Pairs examined, distance calculations	Hand calculation of all 10 pairs
Q5	Reserve boundary (convex hull)	Pairs examined, orientation tests	Andrew's monotone-chain hull
Q6	Helicopter loading (knapsack)	Subsets, item checks, additions	Dynamic programming; hand-listed 16 subsets
Q7	Redesign of Q6	Nodes, candidates, total operations	Same optimum as Q6 and as dynamic programming

2. Q1 — SORTING UNDER DIFFERENT DATA CONDITIONS

2.1 Problem

A fulfilment centre arranges orders by a **dispatch priority score** (here, a distinct integer between 1 and 1,000,000; the program sorts in ascending order). The team is choosing between selection sort, bubble sort and insertion sort. The question is whether one of them should always be preferred, or whether the right choice depends on how the incoming data is arranged.

Three measures are recorded for every run: **key comparisons** (two scores compared), **swaps or shifts** (a swap exchanges two elements; a shift moves one element one place to the right in insertion sort), and **writes** (array slots written — a swap writes two, a shift writes one). Bubble sort is the version with the early-exit flag: it stops after a pass that makes no swap.

2.2 Prediction Before Implementation

Selection sort ignores the input order. Pass i scans the whole unsorted part to find the minimum, whatever the data looks like, so the number of comparisons is fixed. It makes at most one swap per pass, so it never makes more than n − 1 swaps.

$$C_{\text{sel}}(n) = \sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}, \qquad S_{\text{sel}}(n) \le n-1$$

Bubble sort and insertion sort both repair inversions. An inversion is a pair of positions i < j with a[i] > a[j]. Each bubble-sort swap and each insertion-sort shift removes exactly one inversion, so both must perform exactly I moves, where I is the number of inversions in the input. Already-sorted input has I = 0, reverse-sorted input has the maximum I = n(n − 1)/2, and a random order has n(n − 1)/4 on average.

$$S_{\text{bubble}} = S_{\text{insertion}} = I, \qquad 0 \le I \le \frac{n(n-1)}{2}, \qquad E[I_{\text{random}}] = \frac{n(n-1)}{4}$$

Insertion sort's comparisons follow the inversions. Each key is compared once more than it is shifted (the comparison that stops it), except when it travels all the way to the front. So its cost adapts to the data: n − 1 comparisons on sorted input, about I + (n − 1) in general.

$$C_{\text{ins}}(n) \approx I + (n-1), \qquad C_{\text{ins}}^{\text{best}} = n-1, \qquad C_{\text{ins}}^{\text{worst}} = \frac{n(n-1)}{2}$$

Bubble sort's comparisons follow the number of passes, not the number of inversions. One pass can carry a large element all the way to the right, but it moves a small element only one place to the left. A small element near the end of the array (a "turtle") therefore forces almost n − 1 passes, even if it is the only element out of place. My prediction was that bubble

sort would look as good as insertion sort on sorted data, but much worse whenever a small value sits late in the array.

Table 2.1: Predicted Counts Before Running (n = 1,000)

Algorithm	Input	Comparisons	Swaps/shifts	Reason
Selection	sorted	499,500	0	always scans the whole unsorted part
Selection	reverse	499,500	500	same comparisons; about n/2 swaps
Selection	random	499,500	≤ 999	same comparisons; at most n − 1 swaps
Bubble	sorted	999	0	one pass, no swap, early exit
Bubble	reverse	499,500	499,500	every pair is an inversion
Bubble	random	≈ 499,500	≈ 249,750	swaps = inversions ≈ n(n − 1)/4
Insertion	sorted	999	0	each key compared once, never moves
Insertion	reverse	499,500	499,500	each key moves to the front
Insertion	random	≈ 250,749	≈ 249,750	shifts = inversions

2.3 Designed Datasets

The assignment asks for two deliberately contrasting datasets and one edge case of my own. I also added two supporting datasets, one of which is the 95% scenario from the design decision. All five have n = 1,000 distinct scores generated from my seed.

Table 2.2: Datasets and the Behaviour Each One Exposes

Dataset	How it is built	Why it exposes differences
D1 already sorted (contrasting pair)	1,000 scores in ascending order	Zero inversions: the adaptive algorithms (bubble, insertion) should do almost nothing; selection should still do all of its work.
D2 reverse sorted (contrasting pair)	The same scores in descending order	Maximum inversions (499,500): every algorithm reaches its worst case; shows the full cost of moving data.
D3 turtle — own edge case	Sorted, but the most urgent order (the smallest score) is moved to the very end	Only 999 inversions, so the data is "almost sorted" — yet the turtle can move only one place per bubble pass. Separates bubble sort from insertion sort.
D4 random order (supporting)	The scores shuffled	Average case: tests the n(n − 1)/4 inversion prediction.
D5 95% sorted + 5% batch (decision case)	950 orders already in order, followed by a batch of 50 new orders in arrival order	The situation the fulfilment centre reports.

For D5 I predicted the inversion count before running. Each of the k = 50 new orders sits after the 950 sorted ones and is, on average, smaller than half of them; the batch also has about k(k − 1)/4 inversions inside itself:

$$E[I_{D5}] \approx \frac{k(n-k)}{2} + \frac{k(k-1)}{4} = \frac{50 \cdot 950}{2} + \frac{50 \cdot 49}{4} \approx 24{,}363$$

This predicts about 24,363 + 999 ≈ 25,362 comparisons for insertion sort. Because the new orders arrive at the end, many of them are turtles, so I predicted bubble sort would still need close to n(n − 1)/2 comparisons.

2.4 Implementation

Each algorithm works on a copy of the input and returns its counters. Every output is checked against Python's built-in `sorted()`. A separate merge-sort routine counts the inversions of every input independently, so that the theory "swaps = shifts = inversions" can be checked on every dataset.

```
SELECTION-SORT(a)                    BUBBLE-SORT(a)  (early exit)
for i = 0 .. n-2                     for i = 0 .. n-2
    m = i                                swapped = false
    for j = i+1 .. n-1                   for j = 0 .. n-2-i
        comps += 1                           comps += 1
        if a[j] < a[m]: m = j                if a[j] > a[j+1]
    if m != i                                    swap a[j], a[j+1]; swaps += 1
        swap a[i], a[m]; swaps += 1              swapped = true
                                             if not swapped: stop

INSERTION-SORT(a)
for i = 1 .. n-1
    key = a[i]; j = i - 1
    while j >= 0
        comps += 1
        if a[j] > key: a[j+1] = a[j]; shifts += 1; j -= 1
        else: break
    a[j+1] = key
```

2.5 Results

Table 2.3: Measured Counts, n = 1,000 (from results/q1_main.csv)

Dataset	Algorithm	Comparisons	Swaps/ shifts	Writes	Passes	Time (ms)
D1 already sorted	Selection	499,500	0	0	999	24.00
	Bubble	999	0	0	1	0.06
	Insertion	999	0	0	999	0.10
D2 reverse sorted	Selection	499,500	500	1,000	999	17.92
	Bubble	499,500	499,500	999,000	999	65.26
	Insertion	499,500	499,500	500,499	999	40.82
D3 turtle (min at end)	Selection	499,500	999	1,998	999	23.05

Dataset	Algorithm	Comparisons	Swaps/ shifts	Writes	Passes	Time (ms)
	Bubble	499,500	999	1,998	999	31.03
	Insertion	1,997	999	1,000	999	0.19
D4 random order	Selection	499,500	996	1,992	999	22.13
	Bubble	496,497	250,850	501,700	922	43.85
	Insertion	251,843	250,850	251,844	999	34.11
D5 95% sorted + 5% batch	Selection	499,500	952	1,904	999	34.18
	Bubble	496,097	24,462	48,924	917	61.46
	Insertion	25,461	24,462	24,512	999	5.65

The inversion counts measured independently were 0, 499,500, 999, 250,850 and 24,462 for D1–D5. On every dataset, bubble-sort swaps and insertion-sort shifts equalled the inversion count exactly; the program reports this check as PASS (Figure 4).

Figure 1: Key comparisons per dataset, n = 1,000 (log scale)

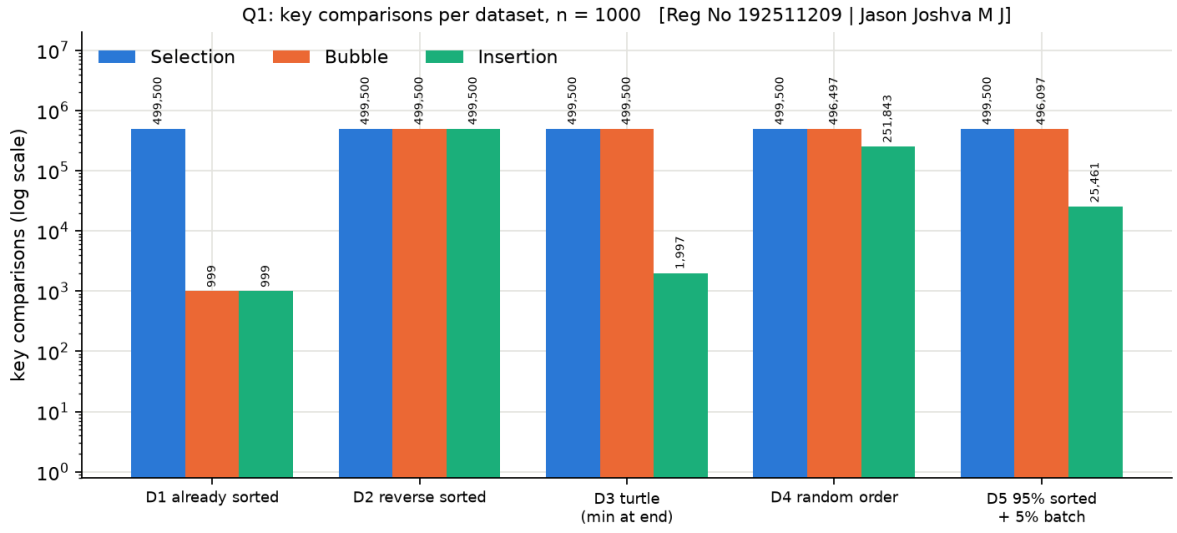


Figure 2: Swaps (selection, bubble) and shifts (insertion) per dataset, n = 1,000 (log scale; zero bars are not drawn)

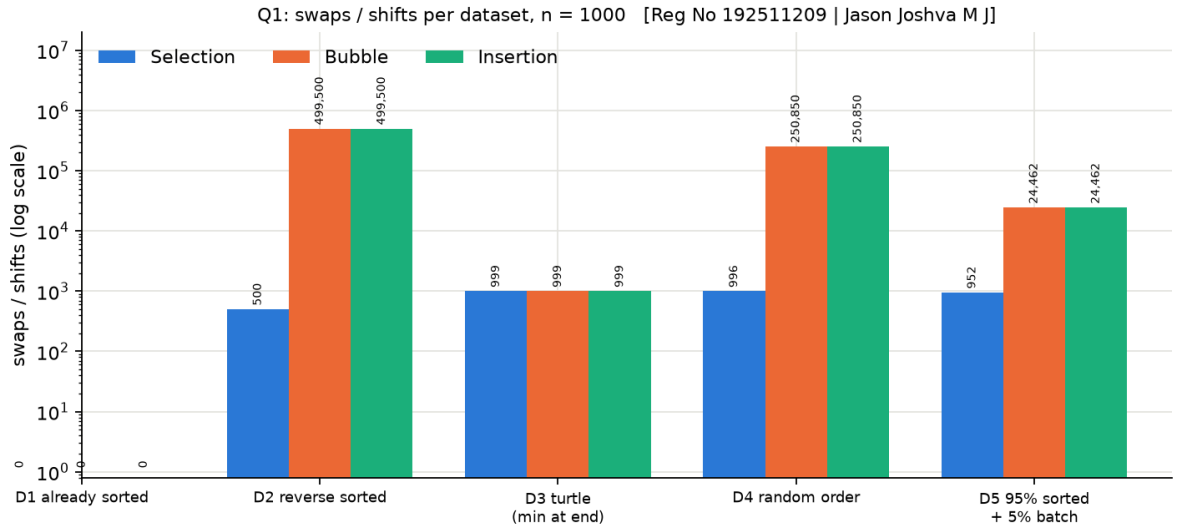


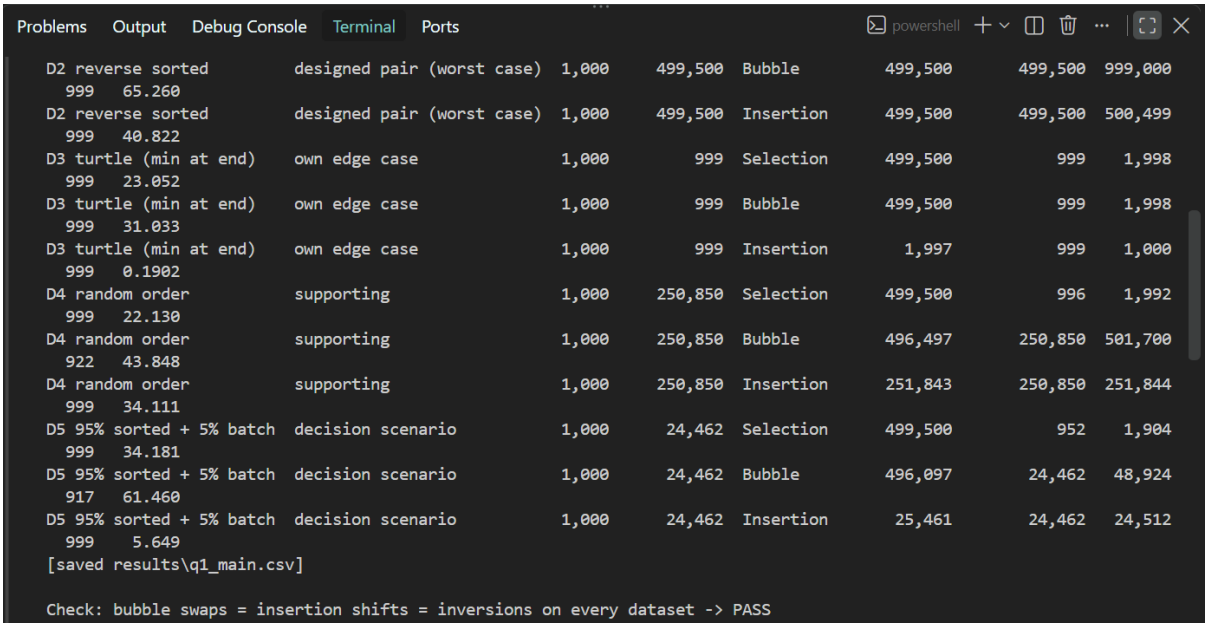
Figure 3: Terminal output of q1_sorting.py — banner, theory table and the start of Experiment 1

```
================================================================================================
 DAA Assignment 2 (CO2)  |  Q1 Sorting under different data conditions
 Reg No: 192511209  |  Name: Jason Joshva M J  |  Seed: 1209 (last 4 digits of reg no)
 Run at: 2026-10-16 21:03:49 (system local time)  |  Python 3.13.14
================================================================================================

--- Theory written BEFORE running (n = 1000) ---------------------------------------------------
  algorithm  input    comparisons  swaps/shifts  reason
  ---------  -------  -----------  ------------  -------------------------------------
  Selection  sorted       499,500             0  always scans the whole unsorted part
  Selection  reverse      499,500           500  same comparisons; ~n/2 swaps
  Selection  random       499,500           999  same comparisons; <= n-1 swaps
  Bubble     sorted           999             0  1 pass, no swap -> early exit
  Bubble     reverse      499,500       499,500  every pair is an inversion
  Bubble     random      ~499,500      ~249,750  swaps = inversions ~ n(n-1)/4
  Insertion  sorted           999             0  each key compared once, never moves
  Insertion  reverse      499,500       499,500  each key moves to the front
  Insertion  random      ~250,749      ~249,750  shifts = inversions

--- Experiment 1: designed datasets, n = 1000 --------------------------------------------------
  dataset                role                       n      inversions  algorithm  comparisons  swaps/shifts  writes
 passes  time_ms
  ---------------------  -------------------------  -----  ----------  ---------  -----------  ------------  -------
 ------  -------
  D1 already sorted      designed pair (best case)  1,000           0  Selection      499,500             0        0
    999   24.002
  D1 already sorted      designed pair (best case)  1,000           0  Bubble             999             0        0
      1   0.0572
  D1 already sorted      designed pair (best case)  1,000           0  Insertion          999             0        0
    999   0.0956
  D2 reverse sorted      designed pair (worst case) 1,000     499,500  Selection      499,500           500    1,000
```

Figure 4: Terminal output — the rest of Experiment 1 and the inversion check (PASS)



2.6 Prediction Compared with Observation

Table 2.4: Predicted Against Observed

Case	Predicted	Observed	Verdict
Selection, every dataset — comparisons	499,500	499,500	Exact
Selection, D2 — swaps	500	500	Exact
Bubble and insertion, D1 — comparisons	999	999	Exact
Bubble and insertion, D2 — swaps/shifts	499,500	499,500	Exact
D4 — inversions	≈ 249,750	250,850	0.44% above the average
D4 insertion — comparisons	≈ 250,749	251,843	Within 0.44%
D5 — inversions	≈ 24,363	24,462	Within 0.5%
D5 insertion — comparisons	≈ 25,362	25,461	Within 0.4%
D3 bubble — passes	close to n − 1	999 passes	Confirmed: worst-case passes
D5 bubble — comparisons	close to 499,500	496,097	Confirmed

Every prediction was supported. The most important result is D3. The data has only 999 inversions — the same order of size as the sorted list — but bubble sort used all 999 passes and the full 499,500 comparisons, exactly as on the reverse-sorted list. Insertion sort handled the same data with 1,997 comparisons, 250 times fewer: it moved the turtle to the front in one insertion of 999 shifts. Selection sort did 499,500 comparisons as always, and on D3 it even

performed a swap in every pass, because each swap pushes the next-smallest element to the end.

Execution time agrees with the counts in direction, but not exactly in size. For example, on D2 bubble and insertion sort make the same 499,500 comparisons, but bubble sort took 65.26 ms against 40.82 ms because each swap writes two array slots and each shift writes one (999,000 against 500,499 writes).

2.7 Growth with n

I repeated three of the conditions at n = 250, 500, 1,000 and 2,000 (Figure 5). Selection sort's comparisons were exactly n(n − 1)/2 on every input (31,125; 124,750; 499,500; 1,999,000), growing about four times when n doubles. On sorted input bubble and insertion sort grew only linearly (249, 499, 999, 1,999). On the 95% sorted data insertion sort needed 103,704 comparisons at n = 2,000, against 1,999,000 for selection sort and 1,998,649 for bubble sort.

Figure 5: Key comparisons against n for random, already-sorted and 95%-sorted data (log–log)

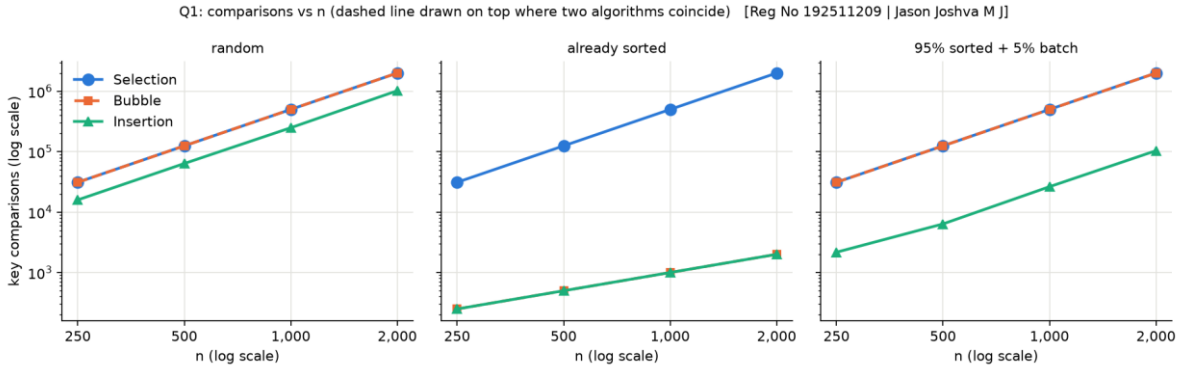


Figure 6: Terminal output — Experiment 2, growth with n

```
--- Experiment 2: growth with n (random, already sorted, 95% sorted + 5% batch) ---------------
  dataset                 n      algorithm  comparisons  swaps/shifts  time_ms
  ----------------------  -----  ---------  -----------  ------------  -------
  random                    250  Selection       31,125           244    3.147
  random                    250  Bubble          31,080        15,662    5.881
  random                    250  Insertion       15,907        15,662    4.173
  already sorted            250  Selection       31,125             0    2.841
  already sorted            250  Bubble             249             0    0.015
  already sorted            250  Insertion          249             0   0.0799
  95% sorted + 5% batch     250  Selection       31,125           247    4.798
  95% sorted + 5% batch     250  Bubble          31,119         1,917    4.393
  95% sorted + 5% batch     250  Insertion        2,166         1,917   0.4856
  random                    500  Selection      124,750           494   11.431
  random                    500  Bubble         124,714        63,100   18.028
  random                    500  Insertion       63,594        63,100   11.856
  already sorted            500  Selection      124,750             0    9.294
  already sorted            500  Bubble             499             0   0.0562
  already sorted            500  Insertion          499             0   0.1051
  95% sorted + 5% batch     500  Selection      124,750           481   10.105
  95% sorted + 5% batch     500  Bubble         124,560         5,869   13.580
  95% sorted + 5% batch     500  Insertion        6,368         5,869   0.9336
  random                  1,000  Selection      499,500           989   23.905
  random                  1,000  Bubble         499,422       247,555   72.589
  random                  1,000  Insertion      248,548       247,555   43.182
  already sorted          1,000  Selection      499,500             0   31.788
  already sorted          1,000  Bubble             999             0   0.0691
  already sorted          1,000  Insertion          999             0   0.1208
  95% sorted + 5% batch   1,000  Selection      499,500           978   38.310
  95% sorted + 5% batch   1,000  Bubble         498,372        25,288   46.346
  95% sorted + 5% batch   1,000  Insertion       26,287        25,288    3.602
```

2.8 Design Decision — 95% of Orders Already in Order

To make the decision robust, I did not rely on the 5% case alone. I varied the size of the newly added batch from 1% to 50% of the 1,000 orders (Table 2.5, Figure 7).

Table 2.5: Sorted Orders Plus a New Batch, n = 1,000 (from results/q1_batch_sweep.csv)

New batch	Selection comparisons	Bubble comparisons	Insertion comparisons	Insertion shifts
1%	499,500	498,797	5,803	4,804
2%	499,500	499,329	11,831	10,833
5%	499,500	499,247	26,162	25,163
10%	499,500	499,247	49,309	48,310
20%	499,500	498,639	93,350	92,351
50%	499,500	499,290	177,304	176,305

Figure 7: Key comparisons when a new batch is appended to sorted orders, n = 1,000



Figure 8: Terminal output — Experiment 3, the design-decision sweep

```
--- Experiment 3: design decision - sorted orders + new batch of size k, n = 1000 -------------
  new batch  n      algorithm  comparisons  swaps/shifts  writes   passes  time_ms
  ---------  -----  ---------  -----------  ------------  -------  ------  -------
  1%         1,000  Selection      499,500           966    1,932     999   31.749
  1%         1,000  Bubble         498,797         4,804    9,608     962   33.880
  1%         1,000  Insertion        5,803         4,804    4,814     999   0.9092
  2%         1,000  Selection      499,500           997    1,994     999   54.082
  2%         1,000  Bubble         499,329        10,833   21,666     981   29.223
  2%         1,000  Insertion       11,831        10,833   10,853     999    1.169
  5%         1,000  Selection      499,500           979    1,958     999   20.007
  5%         1,000  Bubble         499,247        25,163   50,326     977   27.135
  5%         1,000  Insertion       26,162        25,163   25,213     999    2.723
  10%        1,000  Selection      499,500           986    1,972     999   19.553
  10%        1,000  Bubble         499,247        48,310   96,620     977   34.348
  10%        1,000  Insertion       49,309        48,310   48,409     999    6.462
  20%        1,000  Selection      499,500           993    1,986     999   26.069
  20%        1,000  Bubble         498,639        92,351  184,702     958   31.294
  20%        1,000  Insertion       93,350        92,351   92,551     999    8.248
  50%        1,000  Selection      499,500           989    1,978     999   19.296
  50%        1,000  Bubble         499,290       176,305  352,610     979   38.430
  50%        1,000  Insertion      177,304       176,305  176,804     999   33.891
  [saved results\q1_batch_sweep.csv]
  [graph results\graphs\q1_comparisons.png]
  [graph results\graphs\q1_moves.png]
  [graph results\graphs\q1_growth.png]
  [graph results\graphs\q1_batch_sweep.png]

  [q1 finished in 20.2 s]
```

Recommendation: insertion sort. The evidence supports it on three points:

1. **It does work in proportion to the disorder.** Its cost is about n + I comparisons. With a 5% batch it used 25,461 comparisons, about 5% of selection sort's 499,500 and of bubble sort's 496,097. The measured time agrees: 5.65 ms against 34.18 ms and 61.46 ms.

2. **It is not fooled by where the new orders are.** New orders are appended at the end, so some of them have to travel far forward. Bubble sort cannot move them quickly (917 passes on D5, 977 passes with a 5% batch in Table 2.5), so it stays near its worst case at every batch size. Insertion sort moves each new order to its place in a single insertion.
3. **It matches the way the data arrives.** Each new order is simply inserted into an already-sorted list, so the centre can keep the list sorted continuously, at a cost proportional to how far each new order has to move.

Limits of the recommendation. Insertion sort's advantage shrinks as the batch grows, because its shifts are writes. At a 50% batch it still made 2.8 times fewer comparisons than selection sort, but 176,804 writes against 1,978, and in time it was slower (33.89 ms against 19.30 ms). If the incoming data were ever mostly unsorted, or if moving a record were very expensive (for example, physically moving parcels), selection sort's guaranteed $n - 1$ swaps would become the better choice. No algorithm is therefore always preferable: the choice depends on the input's inversions and on the relative cost of comparisons and moves. For the reported 95% condition, insertion sort is clearly best.

3. Q2 — WHEN IS SEQUENTIAL SEARCH AN APPROPRIATE ENGINEERING CHOICE?

3.1 Problem

An aircraft maintenance hangar keeps a temporary list of the specialised tools currently issued. Each record holds **Tool ID, Technician ID, Issue Time and Work Bay**. Records are appended when a tool is issued and removed when it is returned, so the list is kept in issue order and is **not sorted by Tool ID**. To answer "Who currently has Tool T-184?" the system compares the Tool ID of each record with the target, from the first record to the last.

The measured quantity is the number of **key comparisons** (one Tool ID compared with the target). As a first demonstration, in a hangar list of 300 issued tools generated from my seed, T-184 was found at list position 254 after exactly 254 comparisons (Figure 9).

3.2 Prediction

A successful search for the record at position i makes exactly i comparisons. An unsuccessful search must look at every record before it can say "not issued", so it always makes n comparisons. If the target is equally likely to be at any position, the average successful search costs (n + 1)/2. With a share p of successful lookups, the average cost is:

$$C(i) = i, \qquad C_{\text{miss}} = n, \qquad \bar{C}_{\text{hit}} = \tfrac{n+1}{2}, \qquad \bar{C}(p) = p\,\tfrac{n+1}{2} + (1-p)\,n$$

I assumed p = 0.9 (most questions are about tools that really are issued), which gives an average of about 0.55n comparisons per lookup. My prediction was that the cost grows linearly with n, and that unsuccessful searches are the worst case and are about twice as expensive as an average successful one.

3.3 Experiment Design

Table 3.1: Search Experiments

Experiment	What is varied	Predicted comparisons
1. Position of the target	n = 1,000; target at the first, 25%, middle, 75% and last position	1, 250, 500, 750, 1,000
1b. Unsuccessful search	n = 1,000; three Tool IDs that are not in the list	1,000 each
2. List size	n = 100, 1,000, 10,000, 100,000; 100–1,000 random successful and unsuccessful lookups each	(n + 1)/2 and n
3. Operating boundary	Lookup rate from 1 per minute to 10,000 per second	Largest n meeting both rules (Section 3.6)

Experiment	What is varied	Predicted comparisons
4. Changed condition	1 hangar → 20 facilities	Comparisons ×20 per lookup, total work ×400

3.4 Results

Table 3.2: Comparisons Against Target Position, n = 1,000

Search	Position	Predicted	Observed
Successful — first	1	1	1
Successful — 25%	250	250	250
Successful — middle	500	500	500
Successful — 75%	750	750	750
Successful — last	1,000	1,000	1,000
Unsuccessful (T-32243, T-22896, T-40730)	—	1,000	1,000 each

Table 3.3: Comparisons Against List Size (from results/q2_sizes.csv)

n	Lookups	Predicted avg (hit)	Observed avg (hit)	Observed miss	µs per miss	ns per comparison
100	1,000	50.5	49.8	100	6.47	64.7
1,000	1,000	500.5	494.7	1,000	60.58	60.6
10,000	500	5,000.5	5,137.3	10,000	494.64	49.5
100,000	100	50,000.5	49,303.4	100,000	7,953.91	79.5

The successful-search averages are within 3% of (n + 1)/2. The small differences come from sampling: the targets were picked at random, and with 100 lookups on a 100,000-record list the standard error of the mean is about 2,900 comparisons, larger than the observed gap of 697. Every unsuccessful search made exactly n comparisons. Both curves are straight lines with slope 1 on the log–log plot (Figure 10), confirming linear growth.

Figure 9: Terminal output of q2_sequential_search.py — the T-184 lookup and Experiment 1

```
================================================================================================
 DAA Assignment 2 (CO2)  |  Q2 Sequential search - aircraft maintenance tool tracking
 Reg No: 192511209  |  Name: Jason Joshva M J  |  Seed: 1209 (last 4 digits of reg no)
 Run at: 2026-10-16 21:13:12 (system local time)  |  Python 3.13.14
================================================================================================

--- Demo: "Who currently has Tool T-184?"  (one hangar, 300 issued tools) --------------------
  Found at list position 254: ('T-184', 'TECH-417', '09:12', 'BAY-03')  ->  254 comparisons

--- Experiment 1: target position vs comparisons (n = 1000) ----------------------------------
  search                 position  predicted comps  observed comps
  ---------------------  --------  ---------------  --------------
  successful - first            1                1               1
  successful - 25%            250              250             250
  successful - middle         500              500             500
  successful - 75%            750              750             750
  successful - last         1,000            1,000           1,000
  unsuccessful (T-32243)  -                  1,000           1,000
  unsuccessful (T-22896)  -                  1,000           1,000
  unsuccessful (T-40730)  -                  1,000           1,000
```

Figure 10: Left — comparisons against the target's position; right — comparisons against list size (log–log)

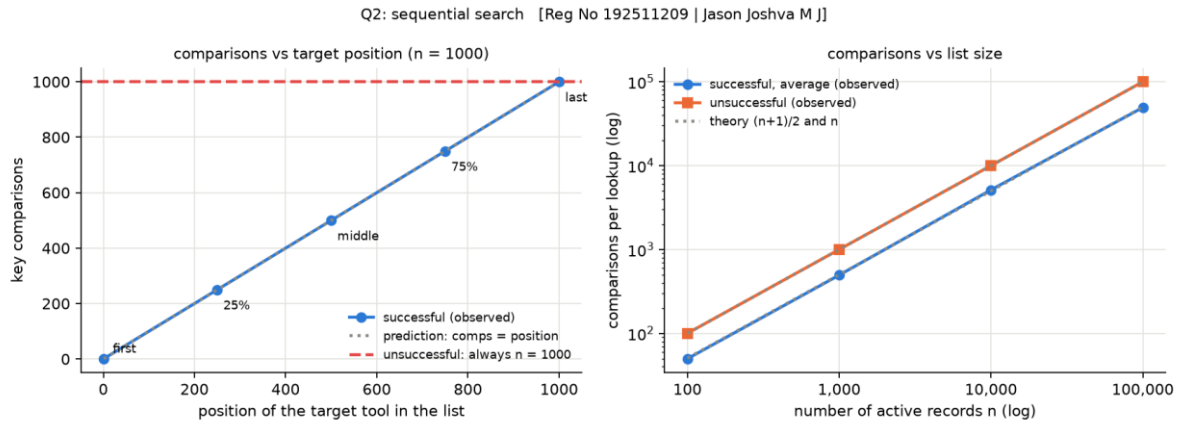


Figure 11: Terminal output — Experiment 2, list size

```
--- Experiment 2: list size vs comparisons (averaged over many random lookups) ---------------
  n        lookups  pred avg hit  obs avg hit  min    max     pred miss  obs miss   us/hit   us/miss  ns/comparison
  -------  -------  ------------  -----------  -----  ------  ---------  ---------  -------  -------  -------------
      100    1,000        50.500       49.758      1     100        100    100.000    2.945    6.472         64.716
    1,000    1,000       500.500      494.684      2   1,000      1,000    1,000.0   26.486   60.582         60.582
   10,000      500       5,000.5      5,137.3     14   9,971     10,000   10,000.0  259.865  494.639         49.464
  100,000      100      50,000.5     49,303.4  1,814  97,161    100,000  100,000.0  5,599.8  7,953.9         79.539
  [saved results\q2_sizes.csv]
```

3.5 Successful Against Unsuccessful Searches

The position of the target is everything for a successful search: the same list gave 1 comparison or 1,000 comparisons depending only on where the tool's record happened to be. An unsuccessful search has no such luck — it always costs n, about twice the average successful search. In this application misses are common (a tool that has just been returned, a mistyped ID), so the **unsuccessful search sets the response time a technician actually experiences in the worst case**. I used it to define the latency rule below.

3.6 Operating Boundary — Dataset Size and Request Frequency

Both the size of the list and the rate of requests matter, so I defined two rules that a sequential search must meet:

1. **Latency rule:** an unsuccessful search (n comparisons) must finish within **1 ms**. A response feels instantaneous to a user when it takes under about 0.1 s; the search is only one step of a request (network, database access, screen update), so I allowed it 1% of that time.
2. **Load rule:** lookups may use at most **5% of one CPU core**, leaving the rest for issue and return transactions and other services.

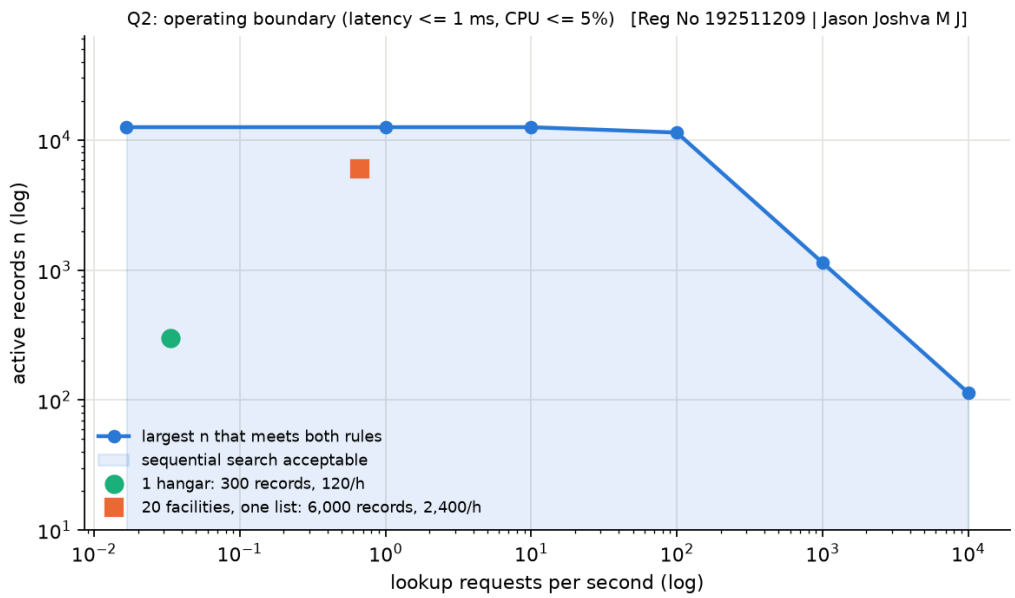
Using the measured cost of one comparison on the largest list (79.5 ns — the most pessimistic value measured), the two rules give:

$$n \cdot t_c \le 1\ \text{ms} \Rightarrow n \le 12{,}572, \qquad r \cdot 0.55\, n \cdot t_c \le 0.05 \Rightarrow n \le \frac{0.05}{0.55\, r\, t_c}$$

Table 3.4: Largest List Size for Which Sequential Search Is Acceptable (from results/q2_boundary.csv)

Lookup rate	n max (latency)	n max (load)	n max	Limited by
1 per minute	12,572	68,576,898	12,572	latency
1 per second	12,572	1,142,948	12,572	latency
10 per second	12,572	114,294	12,572	latency
100 per second	12,572	11,429	11,429	load
1,000 per second	12,572	1,142	1,142	load
10,000 per second	12,572	114	114	load

Figure 12: Operating boundary — the shaded area is where sequential search meets both rules; the points mark the two scenarios of Section 3.7



Sequential search is a reasonable engineering choice when the list is small (below about 12,000 active records at human request rates), changes continuously, and is queried at up to about 10 requests per second. In that region it needs no index, no sorting and no maintenance when tools are issued or returned, and it can never return stale data.

I would stop recommending it when either rule fails: for lists above about 12,500 records (any single miss exceeds 1 ms), or when lookups become frequent — for example automated scanners or dashboards making 1,000 requests per second, where the limit falls to about 1,100 records. Beyond these points an index keyed by Tool ID is needed.

3.7 Changed Condition — From One Hangar to 20 Facilities

With 20 facilities on one shared list, both the number of records and the number of lookups grow by a factor of 20. Each lookup scans 20 times more records, and there are 20 times more lookups, so the total work grows by 20 × 20 = 400. In general, with k facilities the work grows like k^2.

Table 3.5: One Hangar Against 20 Facilities (from results/q2_scenarios.csv)

Scenario	Records	Lookups per hour	Avg comps per lookup	Comps per hour	Worst case (ms)	CPU (%)	Margin (latency)
1 hangar	300	120	165	19,800	0.024	0.00004	×42
20 facilities, one list	6,000	2,400	3,300	7,920,000	0.477	0.0175	×2.1

Figure 13: Terminal output — Experiment 3 (boundary) and the changed condition

```
--- Experiment 3: operating boundary ------------------------------------------------------------
  measured cost of one comparison      : 79.5 ns (from n = 100,000)
  latency rule : unsuccessful search (n comparisons) <= 1 ms  ->  n <= 12,572
  load rule    : lookups/s x 0.55 n x 79.5 ns <= 5% of one core
  lookup rate        lookups/s  n max (latency)  n max (load)  n max   limited by
  -----------------  ---------  ---------------  ------------  ------  ----------
  1 per minute         0.01667           12,572    68,576,898  12,572  latency
  1 per second               1           12,572     1,142,948  12,572  latency
  10 per second             10           12,572       114,294  12,572  latency
  100 per second           100           12,572        11,429  11,429  load
  1,000 per second       1,000           12,572         1,142   1,142  load
  10,000 per second     10,000           12,572           114     114  load
  [saved results\q2_boundary.csv]

--- Changed condition: one hangar -> 20 facilities ----------------------------------------------
  scenario                records  lookups/hour  avg comps/lookup  worst comps  comps/hour  worst ms  CPU s/hour  CPU %
  ----------------------  -------  ------------  ----------------  -----------  ----------  --------  ----------  ---------
  1 hangar                    300           120               165          300      19,800   0.02386    0.001575  4.375e-05
  20 facilities, one list   6,000         2,400             3,300        6,000   7,920,000    0.4772      0.6299     0.0175
  [saved results\q2_scenarios.csv]

  growth from 1 hangar to 20 facilities: comparisons per lookup x20, lookups x20, total work x400
  facilities before the latency rule fails : about 42 (worst case grows like k)
  facilities before the load rule fails    : about 338 (total work grows like k^2)
  hash index alternative: 0.062 us per lookup (about 1 probe), but it must be updated on every issue and return
  [graph results\graphs\q2_comparisons.png]
  [graph results\graphs\q2_boundary.png]

  [q2 finished in 4.7 s]
```

Re-evaluation. The work per hour grew 400 times (19,800 → 7,920,000 comparisons), and the worst-case lookup grew 20 times, from 0.024 ms to 0.477 ms. The 20-facility list is still inside both rules, but the safety margin on the latency rule fell from about ×42 to about ×2.1. The program estimates that the latency rule would fail at about **42 facilities**, and the load rule at about **338** (Figure 13).

Revised recommendation. I would keep sequential search only where it is still clearly safe — each facility's own list of a few hundred tools, which the local technicians query. For the organisation-wide question "who has T-184 anywhere?" on one combined list, I would no longer recommend it: the margin is too small for further growth, peak periods or a slower server. Instead I would add a hash index keyed by Tool ID, which my program measured at 0.062 µs per lookup (about one probe), against 477 µs for a sequential miss on 6,000 records. The index has to be updated on every issue and return, but each update is a constant-time insert or delete, which suits a list that changes continuously.

4. Q3 — ENGINEERING THE BEHAVIOUR OF BRUTE-FORCE STRING MATCHING

4.1 Algorithm and Cost Model

A security operations centre screens system logs for known event signatures. The brute-force matcher tries every alignment ("window") of the pattern against the text: window i compares the pattern with text[i .. i + m − 1] from left to right and stops at the first mismatching character. A screening tool must report every occurrence, so the search does not stop at the first match. For a text of length n and a pattern of length m there are W windows:

$$W = n - m + 1, \qquad W \le C \le W \cdot m$$

Each window costs between 1 comparison (the first character already differs) and m comparisons (a match, or a mismatch at the last character), so the total cost C lies between W and W · m. **Where in this range a test lands depends on how far each window gets before it fails** — that is, on the structure of the text and the pattern, not on n and m alone.

To predict before running, I used a simple **character-frequency model**. Let f(c) be the share of character c in the text. A window reaches pattern position j only if the first j characters all matched, which (if characters were independent) happens with probability $f(p_0) \cdot f(p_1) \cdots f(p_{j-1})$:

$$C \approx W \cdot \sum_{j=0}^{m-1} \prod_{t=0}^{j-1} f(p_t), \qquad \text{random text over } \sigma \text{ letters}: \ \frac{C}{W} = \frac{1-\sigma^{-m}}{1-1/\sigma} \approx \frac{\sigma}{\sigma-1}$$

4.2 Designed Tests and Predictions (Made Before Running)

All main tests use a text of exactly n = 60,000 characters, so that any difference in work comes from structure, not size.

Table 4.1: Test Design and Prediction

Test	Text and pattern	Prediction	Reason
T1 (little work)	Realistic server log; signature `<script>alert(` (m = 14)	≈ W = 59,987 (1.0 per window)	The first pattern character < almost never occurs in the log, so nearly every window fails on its first comparison.
T2 (engineered worst case)	Text "AAAA…A"; pattern "AAAAAAAAAAAAAB" (m = 14)	W · m = 839,818 exactly	Every window matches 13 characters and fails only at the last one, and the same happens at every one of the 59,987 positions.

Test	Text and pattern	Prediction	Reason
T3 (own test)	Packet hex dump with NOP sleds (long runs of `90`); signature `90 90 90 90 90 90 90 90 31 C0` (m = 29)	Frequency model: 68,385. Structure-aware: 185,516	Inside a sled every window starting at a `9` matches 24 characters before it fails, so long runs of repeated text should multiply the work.
S1 (supporting)	Attack log, 60% of lines "Failed password for invalid user …"; signature "Failed password for root" (m = 24)	Structure-aware: 66,972	Each near-miss line matches the 20-character prefix "Failed password for " before failing.
S2 (supporting)	Same signature on a normal log (3% failed logins)	Structure-aware: 60,421	Few near misses, so close to one comparison per window.
S3 (supporting)	Random A–Z text; random pattern (m = 14, same n and m as T2)	≈ 1.04 per window = 62,411	With 26 letters a window survives its first comparison only 1 time in 26.

For T3 I made two predictions on purpose. The frequency model treats every character independently. The structure-aware prediction instead counts the positions where eight complete `90` groups follow (5,231 positions in my dump) and adds the 24 matched characters each of them costs. Comparing the two shows whether character frequencies alone are enough to predict the work.

4.3 Implementation

```
BRUTE-FORCE-MATCH(text, pattern)          # all occurrences
n = length(text); m = length(pattern); found = []; comps = 0
for i = 0 .. n - m                        # every window
    j = 0
    while j < m
        comps += 1                        # one character comparison
        if text[i + j] != pattern[j]: break
        j += 1
    if j == m: append i to found
return found, comps
```

Every list of match positions was checked against an independent scan using Python's `str.startswith` at every position; all tests report "positions verified" (Figure 14).

4.4 Results

Table 4.2: Character Comparisons for Each Test (from results/q3_tests.csv)

Test	m	Matches	Predicted (freq.)	Predicted (structure)	Observed	Per window	% of worst case W·m
T1	14	3	59,993	—	60,029	1.001	7.15%
T2	14	0	839,818	839,818	839,818	14.000	100.00%
T3	29	10	68,385	185,516	190,615	3.178	10.96%
S1	24	5	60,335	66,972	66,972	1.117	4.65%
S2	24	8	59,998	60,421	60,421	1.007	4.20%

Test	m	Matches	Predicted (freq.)	Predicted (structure)	Observed	Per window	% of worst case W·m
S3	14	0	62,411	—	62,406	1.040	7.43%

Figure 14: Terminal output of q3_string_matching.py — predictions and results for T1, T2, T3 and S1

```
================================================================================
 DAA Assignment 2 (CO2)  |  Q3 Brute-force string matching - cybersecurity log screening
 Reg No: 192511209  |  Name: Jason Joshva M J  |  Seed: 1209 (last 4 digits of reg no)
 Run at: 2026-10-16 21:15:57 (system local time)  |  Python 3.13.14
================================================================================

--- Designed tests (prediction printed before each run) ------------------------------------

  T1: little work: rare first character in a normal log
    text n = 60,000   pattern m = 14  '<script>alert('   windows n-m+1 = 59,987
    PREDICTION (character-frequency model) : about 59,993 comparisons  (1.000 per window)
    OBSERVED                               : 60,029 comparisons (1.001 per window), 3 match(es), positions verified

  T2: engineered worst case: text AAAA..., pattern AAA...AB
    text n = 60,000   pattern m = 14  'AAAAAAAAAAAAAB'   windows n-m+1 = 59,987
    PREDICTION (character-frequency model) : about 839,818 comparisons  (14.000 per window)
    PREDICTION (structure-aware)           : about 839,818 comparisons  = (n-m+1) x m, every window fails at the last character
    OBSERVED                               : 839,818 comparisons (14.000 per window), 0 match(es), positions verified

  T3: own test: NOP-sled signature in a packet hex dump (long runs of '90 ')
    text n = 60,000   pattern m = 29  '90 90 90 90 90 90 90 90 31 C0'   windows n-m+1 = 59,972
    PREDICTION (character-frequency model) : about 68,385 comparisons  (1.140 per window)
    PREDICTION (structure-aware)           : about 185,516 comparisons  = windows + 5,231 sled positions x 24 matched characters
    OBSERVED                               : 190,615 comparisons (3.178 per window), 10 match(es), positions verified

  S1: near-miss prefix: attack log, 60% 'Failed password for invalid user'
    text n = 60,000   pattern m = 24  'Failed password for root'   windows n-m+1 = 59,977
    PREDICTION (character-frequency model) : about 60,335 comparisons  (1.006 per window)
    PREDICTION (structure-aware)           : about 66,972 comparisons  = windows + 344 near misses x 20 + 5 hits x 23
    OBSERVED                               : 66,972 comparisons (1.117 per window), 5 match(es), positions verified
```

Figure 15: Terminal output — S2, S3 and the summary table, followed by Sweep 1

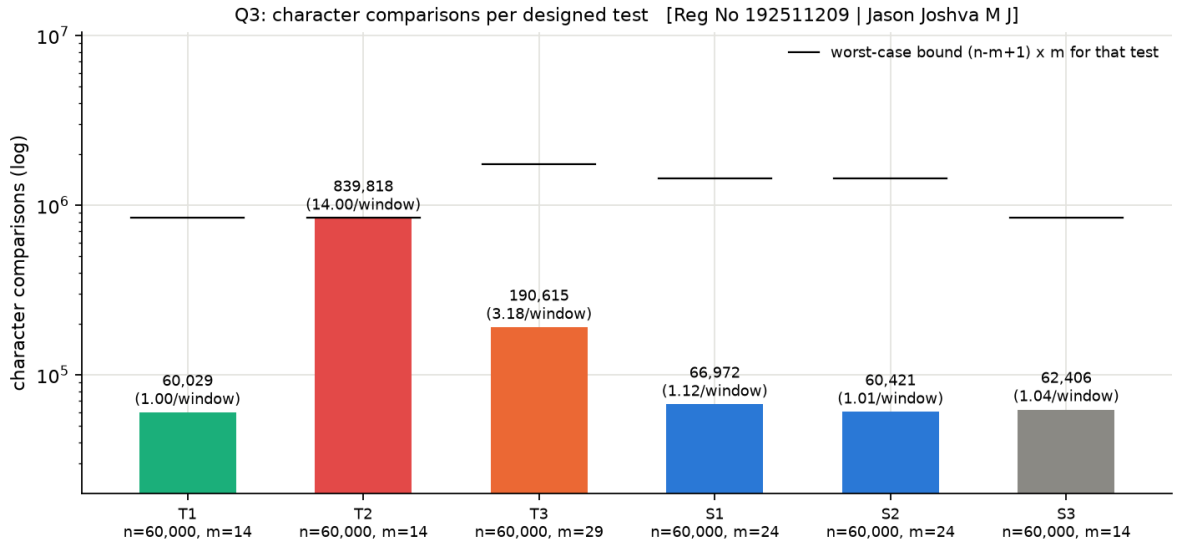
```
  S2: same pattern, normal log (3% failed logins) for contrast
    text n = 60,000   pattern m = 24  'Failed password for root'   windows n-m+1 = 59,977
    PREDICTION (character-frequency model) : about 59,998 comparisons  (1.000 per window)
    PREDICTION (structure-aware)           : about 60,421 comparisons  = windows + 13 near misses x 20 + 8 hits x 23
    OBSERVED                               : 60,421 comparisons (1.007 per window), 8 match(es), positions verified

  S3: same n and m as T2, but random A-Z text
    text n = 60,000   pattern m = 14  'WNHMKYBNCTXIUT'   windows n-m+1 = 59,987
    PREDICTION (character-frequency model) : about 62,411 comparisons  (1.040 per window)
    OBSERVED                               : 62,406 comparisons (1.040 per window), 0 match(es), positions verified

  test  n       m   pattern                    windows  matches  pred (freq)  pred (structure)  observed comps  comps/window  worst (n-m+1)m  % of worst
  ----  ------  --  --------------------------  -------  -------  -----------  ----------------  --------------  ------------  --------------  ----------
  T1    60,000  14  <script>alert(               59,987        3       59,993  -                         60,029         1.001         839,818       7.148
  T2    60,000  14  AAAAAAAAAAAAAB               59,987        0      839,818          839,818         839,818        14.000         839,818     100.000
  T3    60,000  29  90 90 90 90 90 90 90 90 3...  59,972       10       68,385          185,516         190,615         3.178       1,739,188      10.960
  S1    60,000  24  Failed password for root      59,977        5       60,335           66,972          66,972         1.117       1,439,448       4.653
  S2    60,000  24  Failed password for root      59,977        8       59,998           60,421          60,421         1.007       1,439,448       4.198
  S3    60,000  14  WNHMKYBNCTXIUT               59,987        0       62,411  -                         62,406         1.040         839,818       7.431
  [saved results\q3_tests.csv]

--- Sweep 1: worst-case text 'A' x 20,000, pattern 'A'x(m-1)+'B', vary m ----------------------
  n       m   windows  predicted (n-m+1)m  observed   comps per text char
  ------  --  -------  ------------------  ---------  -------------------
  20,000   2   19,999              39,998     39,998                2.000
  20,000   5   19,996              99,980     99,980                4.999
  20,000  10   19,991             199,910    199,910                9.995
  20,000  20   19,981             399,620    399,620               19.981
  20,000  40   19,961             798,440    798,440               39.922
  20,000  80   19,921           1,593,680  1,593,680               79.684
  [saved results\q3_sweep_pattern_length.csv]
```

Figure 16: Character comparisons for each test (log scale); the black bar marks the worst-case bound W·m for that test



4.5 Prediction Compared with the Result

- **T1 — supported.** 60,029 comparisons against a prediction of about 59,993, i.e. 1.001 comparisons per window. Almost all of the 36 extra comparisons come from the three planted attacks, where the whole 14-character signature matched; the frequency model treats those characters as independent and expects such long matches to be very rare.
- **T2 — supported exactly.** 839,818 = 59,987 × 14. The text and pattern were engineered so that every window does the maximum possible work and still finds nothing. With the same n and m as T1, T2 needed **14.0 times more work**.
- **T3 — the frequency model failed; the structure-aware prediction succeeded.** The frequency model predicted 68,385 comparisons, but the result was 190,615, 2.8 times more. The model assumes each character is independent of the one before it, but inside a sled the text repeats `90` with near certainty, so a window that matches its first `9` almost always continues for 24 characters. The structure-aware count (185,516) was within 2.7%. The remaining 5,099 comparisons come from the end of each of the 39 sleds in the dump, where windows match fewer than eight groups (3 to 21 characters, about 3,400 comparisons in total), from payload bytes that happen to start with 9, and from the 10 true matches.
- **S1 and S2 — supported exactly.** The same pattern on two logs of the same length cost 66,972 and 60,421 comparisons. The attack log contained 344 near-miss lines, each costing 20 extra comparisons, which explains the 11% difference exactly.

- **S3 — supported.** Random A–Z text gave 1.040 comparisons per window, as predicted by $\sigma/(\sigma - 1) = 26/25$.

4.6 Sweeps — Pattern Length and Alphabet Size

Table 4.3: Worst-Case Text, n = 20,000, Varying m

m	Windows	Predicted (n−m+1)·m	Observed	Per window	Per text character
2	19,999	39,998	39,998	2	2.000
5	19,996	99,980	99,980	5	4.999
10	19,991	199,910	199,910	10	9.995
20	19,981	399,620	399,620	20	19.981
40	19,961	798,440	798,440	40	39.922
80	19,921	1,593,680	1,593,680	80	79.684

Table 4.4: Random Text, n = 60,000, m = 14, Varying the Alphabet

Alphabet size	Letters	Theory per window	Observed per window	Observed comparisons
2	AB	2.000	2.001	120,015
4	ABCD	1.333	1.334	80,021
26	A–Z	1.040	1.040	62,378

In the worst case the work grows linearly with m: doubling the pattern length doubled the comparisons, so an 80-character signature cost almost 80 comparisons per text character. With random text the work depends on the alphabet: a two-letter alphabet doubles the work compared with a 26-letter one, because half of all windows survive their first comparison.

Figure 17: Left — worst-case work against pattern length; right — comparisons per window against alphabet size

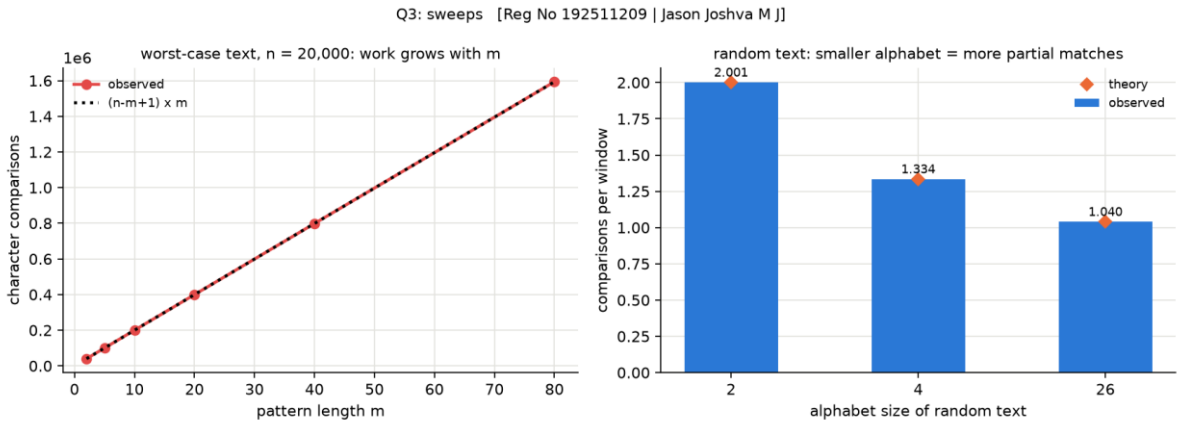


Figure 18: Terminal output — Sweep 2 (alphabet size)

```
--- Sweep 2: random text, n = 60,000, m = 14, vary alphabet size ------------------------------
  alphabet size  letters  windows  theory per window  observed per window  observed comps  matches
  -------------  -------  -------  -----------------  -------------------  --------------  -------
              2  AB        59,987              2.000                2.001         120,015        6
              4  ABCD      59,987              1.333                1.334          80,021        0
             26  A-Z       59,987              1.040                1.040          62,378        0
  [saved results\q3_sweep_alphabet.csv]
  [graph results\graphs\q3_tests.png]
  [graph results\graphs\q3_sweeps.png]

  [q3 finished in 2.3 s]
```

4.7 Inquiry

Why one input requires substantially more work than another. The cost of a window equals the length of the longest prefix of the pattern that matches there, plus one. T1 and T2 have the same n and m, but in T1 almost every window fails at once (1.001 comparisons per window), while in T2 every window travels 13 characters before failing (14 per window). The total work is the sum of these partial-match lengths over all windows.

Which characteristics create repeated comparisons. Repeated work appears when many windows match a long prefix of the pattern: (1) a text made of long runs of the same characters (T2, the NOP sleds of T3); (2) a pattern that begins with a long, common prefix of the text ("Failed password for " in S1); (3) a small alphabet (Table 4.4); and (4) long patterns, which raise the ceiling W · m. In every case the algorithm re-reads characters it has already compared in the previous window, because brute force keeps no memory of what it has seen.

What this reveals about using only input size. Input size gives only the range W to W · m — here 59,987 to 839,818, a factor of 14. Six tests with the same n = 60,000 landed at 4% to 100% of that worst case. For log screening this matters: an attacker who controls part of the log (long padding, repeated bytes) can push the scanner towards its worst case. An algorithm

must be judged by its best, worst and typical behaviour on the inputs it will really meet, not by n alone.

5. Q4 — CLOSEST PAIR AND THE COST OF EXAMINING EVERY POSSIBILITY

5.1 Design Before Implementation

Each drone reports a position $D_i = (x_i, y_i)$ in metres inside a 10 km × 10 km delivery zone. The monitor must find the two drones closest to each other.

- **Which pairs must be examined.** Brute force has no information that rules any pair out, so every pair of distinct drones is a candidate. A drone is never paired with itself.
- **Avoiding duplicate comparisons.** The distance from D_i to D_j equals the distance from D_j to D_i, so each unordered pair is examined only once, by letting j start at i + 1.
- **How many unique pairs exist.** Drone 1 is paired with n − 1 later drones, drone 2 with n − 2, and so on, down to drone n − 1 with one:

$$P(n) = \sum_{i=1}^{n-1} (n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \binom{n}{2} \in \Theta(n^2)$$

Each pair needs one distance calculation, so the number of distance calculations is also n(n − 1)/2. To make each calculation cheap, the program compares **squared distances** ($dx^2 + dy^2$). The square root is increasing, so the order of distances is unchanged, and only one square root is taken at the end.

```
CLOSEST-PAIR-BF(D[0 .. n-1])
best = infinity; pairs = 0
for i = 0 .. n-2
    for j = i+1 .. n-1                    # each unordered pair once
        pairs += 1
        dx = D[i].x - D[j].x; dy = D[i].y - D[j].y
        d2 = dx*dx + dy*dy                # one distance calculation
        if d2 < best: best = d2; (a, b) = (i, j)
return (a, b, sqrt(best), pairs)
```

5.2 Correctness Test

Five drones with small integer coordinates were chosen so that all ten distances can be checked by hand (Table 5.1).

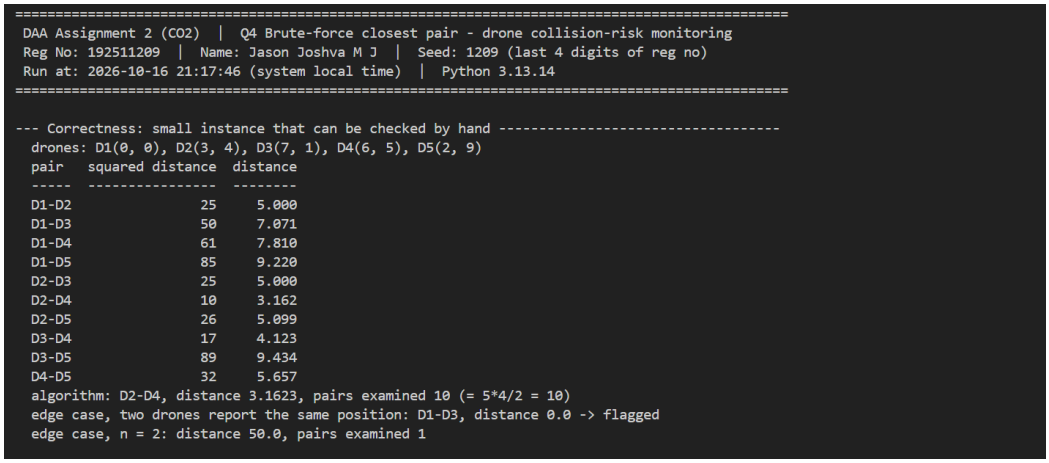
Table 5.1: Hand-Checkable Instance — D1(0,0), D2(3,4), D3(7,1), D4(6,5), D5(2,9)

Pair	Squared distance	Distance
D1–D2	25	5.000
D1–D3	50	7.071
D1–D4	61	7.810
D1–D5	85	9.220

Pair	Squared distance	Distance
D2–D3	25	5.000
D2–D4	**10**	**3.162**
D2–D5	26	5.099
D3–D4	17	4.123
D3–D5	89	9.434
D4–D5	32	5.657

The program returned D2–D4 at distance 3.1623 after examining 10 pairs (= 5 × 4/2), which agrees with the hand calculation. Two edge cases were also handled: two drones reporting the same position were returned with distance 0 and flagged, and with n = 2 exactly one pair was examined (Figure 19).

Figure 19: Terminal output of q4_closest_pair.py — the hand-checkable test and the edge cases



5.3 Growth Experiment

Drone positions were generated uniformly (to the centimetre) for six sizes, each double the previous one.

Table 5.2: Growth with the Number of Drones (from results/q4_growth.csv)

n	n(n−1)/2	Pairs examined	Distance calcs	Closest (m)	Time (ms)	ns per pair	Pairs with |dx| ≥ d_min
125	7,750	7,750	7,750	54.92	1.93	249.5	98.90%
250	31,125	31,125	31,125	12.48	5.73	184.1	99.71%
500	124,750	124,750	124,750	18.27	18.45	147.9	99.64%
1,000	499,500	499,500	499,500	4.54	86.17	172.5	99.91%
2,000	1,999,000	1,999,000	1,999,000	3.06	223.06	111.6	99.94%

n	n(n−1)/2	Pairs examined	Distance calcs	Closest (m)	Time (ms)	ns per pair	Pairs with |dx| ≥ d_min
4,000	7,998,000	7,998,000	7,998,000	3.65	793.40	99.2	99.93%

Theory against experiment. The pairs examined and the distance calculations equal n(n − 1)/2 exactly at every size, and they grew ×4.02, ×4.01, ×4.00, ×4.00 and ×4.00 each time n doubled, as a Θ(n²) algorithm should. Time grew strongly as well (from 1.93 ms to 793.40 ms), but less regularly: while the pairs grew 1,032 times, the time grew 411 times, and the doubling ratios scatter between ×2.59 and ×4.67. The time per pair fell from 249.5 ns to 99.2 ns as n grew, because fixed costs such as the function call and loop start-up are spread over more pairs in a larger run. This is why the counts, not the times, are the reliable evidence.

Figure 20: Left — pairs examined against n with the n(n − 1)/2 curve; right — measured time with the 100 ms budget

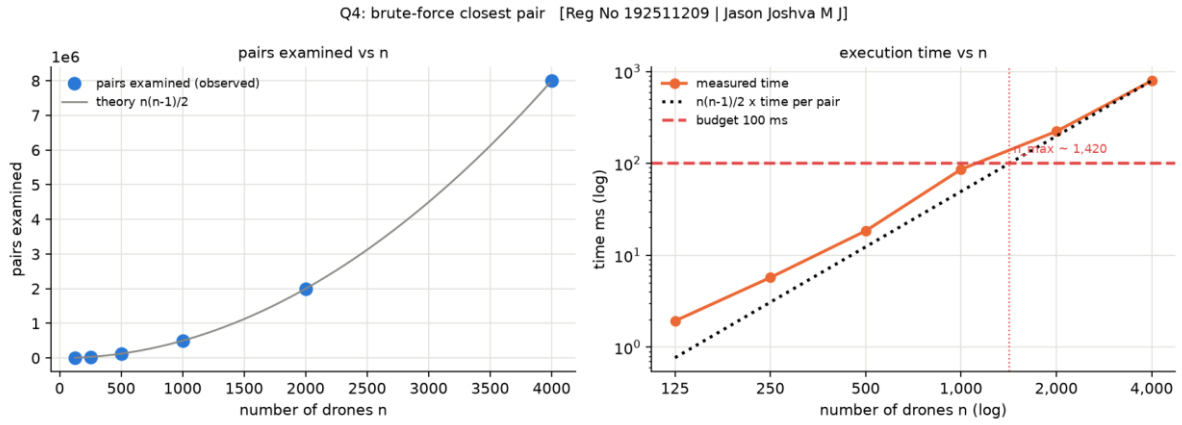


Figure 21: Terminal output — Experiment 1 and the doubling-ratio check

```
--- Experiment 1: growth with the number of drones (uniform positions) ----------------------
  n      theory n(n-1)/2  pairs examined  distance calcs  closest pair  distance m  time ms  ns per pair  % pairs avoidable
-----  ---------------  --------------  --------------  ------------  ----------  -------  -----------  -----------------
  125            7,750           7,750           7,750  D2-D6             54.920    1.933      249.463              98.903
  250           31,125          31,125          31,125  D28-D51           12.480    5.732      184.148              99.708
  500          124,750         124,750         124,750  D117-D344         18.270   18.455      147.934              99.640
1,000          499,500         499,500         499,500  D207-D804          4.540   86.175      172.522              99.909
2,000        1,999,000       1,999,000       1,999,000  D1039-D1174        3.060  223.059      111.585              99.938
4,000        7,998,000       7,998,000       7,998,000  D1790-D2134        3.650  793.395       99.199              99.927
[saved results\q4_growth.csv]

ratio check when n doubles (theory: pairs about x4):
  n   125 ->   250: pairs x4.02, time x2.96
  n   250 ->   500: pairs x4.01, time x3.22
  n   500 ->  1000: pairs x4.00, time x4.67
  n  1000 ->  2000: pairs x4.00, time x2.59
  n  2000 ->  4000: pairs x4.00, time x3.56
```

The work does not depend on where the drones are. With n = 1,000 drones arranged uniformly, in five clusters around depots, or along one straight corridor, the algorithm examined 499,500 pairs every time (Table 5.3). Unlike sorting or string matching, brute-force closest pair has no best or worst case: it always does all of its work. In the corridor test two

drones reported exactly the same position (to the centimetre), which the algorithm correctly reported as a distance of 0.

Table 5.3: Same n, Different Arrangements (from results/q4_arrangements.csv)

Arrangement	Pairs examined	Distance calcs	Closest (m)	Time (ms)
Uniform	499,500	499,500	7.57	78.77
Clustered (5 depots)	499,500	499,500	1.41	59.21
One corridor	499,500	499,500	0.00	55.14

5.4 Scalability — When Does Brute Force Become Impractical?

Acceptable response time: 100 ms. I assumed the drones report their positions about once per second. The collision check is only the first step of the response: avoidance commands still have to be computed and transmitted inside the same cycle, so I allowed the check 10% of the cycle. Using the measured time per pair on the largest run (99.2 ns):

$$\frac{n(n-1)}{2} \cdot t_{\text{pair}} \leq 0.1 \text{ s} \quad \Rightarrow \quad n_{\max} \approx \frac{1+\sqrt{1+8 \cdot 0.1/t_{\text{pair}}}}{2} \approx 1{,}420 \text{ drones}$$

Table 5.4: Predicted Response Time (from results/q4_boundary.csv)

Drones	Pairs	Predicted (ms)	Within 100 ms?
500	124,750	12.4	yes
1,000	499,500	49.6	yes
1,420	1,007,490	99.9	yes (limit)
2,000	1,999,000	198.3	no
5,000	12,497,500	1,239.7	no
10,000	49,995,000	4,959.5	no
50,000	1,249,975,000	123,996.5	no (about 2 minutes)

The measurements agree with this boundary: 1,000 drones took 86.17 ms (inside the budget) and 2,000 drones took 223.06 ms (more than twice the budget). Because the time per pair at n = 1,000 was higher (172.5 ns), the same rule gives about 1,080 drones from that run, so I put the practical limit at **about 1,000–1,400 drones**. Beyond that, every doubling of the fleet multiplies the response time by four, and a fleet of 10,000 drones would need about 5 seconds per check — far too slow for collision avoidance.

Figure 22: Terminal output — arrangements and the scalability calculation

```
--- Experiment 2: same n = 1000, different arrangements (is the work input-sensitive?) --------
  arrangement          n      pairs examined  distance calcs  closest distance m  time ms
  -------------------  -----  --------------  --------------  ------------------  -------
  uniform              1,000         499,500         499,500               7.570   78.775
  clustered (5 depots) 1,000         499,500         499,500               1.410   59.210
  one corridor         1,000         499,500         499,500                   0   55.142
  [saved results\q4_arrangements.csv]

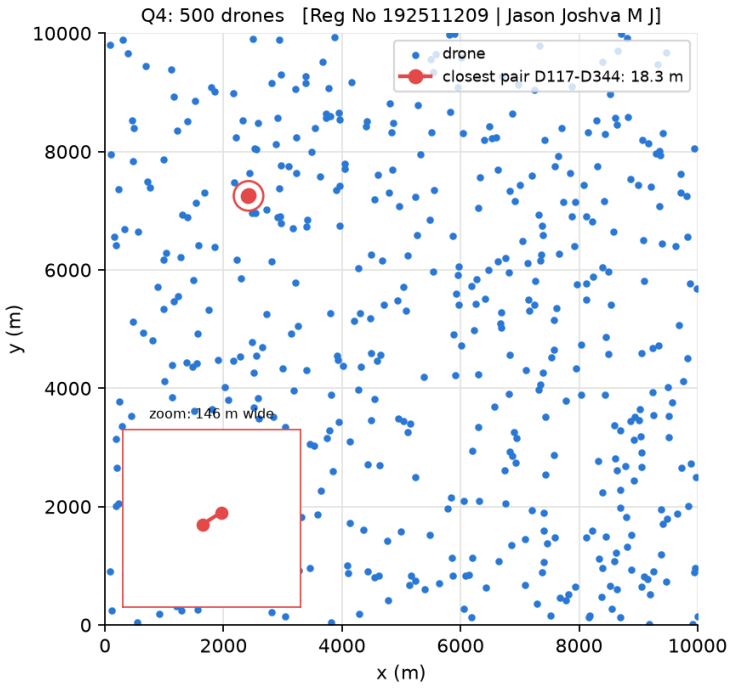
--- Scalability: acceptable response time = 100 ms -----------------------------------------
  measured time per pair (largest run, n = 4,000) : 99.2 ns
  n(n-1)/2 x 99.2 ns <= 0.1 s  ->  n_max = about 1,420 drones
  drones  pairs          predicted ms  within 100 ms?
  ------  -------------  ------------  --------------
     500        124,750        12.375  yes
   1,000        499,500        49.550  yes
   1,420      1,007,490        99.942  yes
   2,000      1,999,000       198.299  NO
   5,000     12,497,500       1,239.7  NO
  10,000     49,995,000       4,959.5  NO
  50,000  1,249,975,000     123,996.5  NO
  [saved results\q4_boundary.csv]
  [graph results\graphs\q4_growth.png]
  [graph results\graphs\q4_drones_n500.png]

  [q4 finished in 4.2 s]
```

5.5 Where the Inefficiency Comes From

The inefficiency comes from computing all $n(n-1)/2$ distances when only pairs that are close together can possibly be the answer. To measure this, the program counted how many pairs had x-coordinates that differ by at least the final closest distance d_min. Such a pair can never be the closest pair, but brute force still computed its full distance. They were **99.9% of all pairs** for $n \geq 1{,}000$ (Table 5.2). At $n = 4{,}000$ the algorithm made almost 8 million distance calculations to find one pair 3.65 m apart.

Figure 23: 500 drone positions with the closest pair (D117–D344, 18.3 m) circled and shown in the zoomed inset



What a redesign must avoid. A better strategy must not compare drones that are obviously far apart. It needs some ordering or spatial structure — for example, sorting the drones by x once ($\Theta(n \log n)$) so that a pair can be rejected as soon as $|dx| \geq$ the best distance found so far, or dividing the zone into grid cells so that each drone is compared only with drones in its own and neighbouring cells. Either way, the number of distance calculations must grow much more slowly than n^2, ideally close to linearly. Using information already gathered (the best distance so far) to skip pairs is exactly what brute force does not do.

6. Q5 — DERIVING A CONVEX HULL ALGORITHM FROM A GEOMETRIC PRINCIPLE

6.1 From the Principle to a Computational Procedure

Monitoring stations $P_i = (x_i, y_i)$ are placed throughout a wildlife reserve, and the outer boundary formed by them is required. The only starting point is the principle that a line joining two candidate points may form part of the outer boundary if all remaining points lie on one side of it. I converted this into five design decisions.

(a) How two points define a line. Two distinct stations P_i and P_j define the directed line from P_i to P_j. Each unordered pair is considered once ($j > i$), since the line through P_j and P_i is the same line.

(b) How the position of another point is determined. The sign of the cross product of the vectors $P_i \rightarrow P_j$ and $P_i \rightarrow P_k$ tells on which side of the line P_k lies:

$$\text{cross}(P_i, P_j, P_k) = (x_j - x_i)(y_k - y_i) - (y_j - y_i)(x_k - x_i)$$

- cross > 0: P_k is to the **left** of the directed line $P_i \rightarrow P_j$;
- cross < 0: P_k is to the **right** of it;
- cross = 0: P_k is **on the line** (collinear).

This value is twice the signed area of the triangle $P_iP_jP_k$. Station coordinates are whole metres, so the cross product is computed **exactly in integers**, with no rounding error in the "= 0" case.

(c) How the remaining points are examined. For each pair, every other station is tested once ($n - 2$ orientation tests), counting how many lie strictly left, strictly right, and on the line.

(d) The condition that identifies a hull edge. The pair P_iP_j is accepted when **no station is strictly left while another is strictly right**, and **every collinear station lies between the two endpoints** P_i and P_j. The second part is my own addition, made after studying collinear points: if a station lies on the line but beyond P_j, then P_iP_j is only part of a longer edge, and only the longest segment (between the two extreme stations) should be kept. Duplicate station reports are removed before the search, because two identical points do not define a line.

(e) How the final boundary is built. Each accepted edge joins two corner stations, and on a proper hull every corner belongs to exactly two accepted edges. Starting from the lowest-leftmost corner, the procedure repeatedly walks to the neighbour it did not just come from, until it returns to the start. The result is then ordered counter-clockwise using the sign of its

area. If every station is collinear, exactly one edge is accepted and the boundary is a line segment.

6.2 Pseudocode

```
BRUTE-FORCE-HULL(S)
P = distinct stations of S, sorted            # remove duplicate reports
E = empty list of edges
for i = 0 .. n-2
    for j = i+1 .. n-1                         # each candidate line once
        left = right = beyond = 0
        for k = 0 .. n-1, k != i, k != j
            c = CROSS(P[i], P[j], P[k])        # one orientation test
            if c > 0: left += 1
            else if c < 0: right += 1
            else if P[k] is not between P[i] and P[j]: beyond += 1
        if not (left > 0 and right > 0) and beyond == 0
            add (i, j) to E                    # P[i]P[j] is a hull edge
return BUILD-BOUNDARY(P, E)

BUILD-BOUNDARY(P, E)
if E has one edge: return its two end points  # all stations collinear
if some corner is not in exactly two edges: report an error
start at the smallest corner; move to the neighbour not just visited
stop when the start is reached; reverse if the signed area is negative
```

6.3 Test Configurations

Table 6.1: Test Design

Test	Configuration	What it challenges
A — random	30 stations at random integer positions in a 20 km × 15 km reserve	The normal case, with both boundary and interior points.
B — designed challenge	A 12 km × 8 km rectangular fence; 8 stations on the fence between the corners; 2 duplicate reports; 4 interior stations on one straight line; 3 other interior stations; 1 station 1 m outside the right fence at (12001, 6000)	Collinear boundary points (the naive rule accepts every sub-segment); duplicates (zero-length lines); an interior collinear line that must not create edges; a corner only 1 m outside the fence, which only exact arithmetic detects.
C — degenerate	12 stations along one straight river bank	Every triple is collinear, so the "hull" is a line segment.
Edge cases	1 station; 2 stations; a triangle; 3 collinear; 4 with one inside; 5 copies of one station	The smallest inputs, where loops run zero or one time.

6.4 Results and Visualisation

Table 6.2: Results (from results/q5_tests.csv)

Test	Stations (distinct)	Pairs	Tests	Accepted edges	Corners	Interior	Check
A — random	30 (30)	435	12,180	9	9	21	PASS
B — challenge	22 (20)	190	3,420	5	5	8	PASS

Test	Stations (distinct)	Pairs	Tests	Accepted edges	Corners	Interior	Check
C — collinear	12 (12)	66	660	1	2	0	PASS
B with naive rule	22 (20)	—	—	24	—	—	FAIL

Every result matched an independent monotone-chain hull. In test B the five corners were (12000, 0), (12001, 6000), (12000, 8000), (0, 8000) and (0, 0). The station 1 m outside the fence became a corner, and because of it the fence station at (12000, 4000) became an interior point — a difference of one metre over a 12 km reserve, detected exactly. The 7 other fence stations were reported as lying on the boundary (12 boundary stations in total). In test C the boundary was the segment from (1000, 2000) to (17500, 13000), with all 12 stations on it. All six edge cases passed (Figure 27).

Figure 24: Boundaries found for tests A, B and C (corners in red, stations lying on an edge in orange, interior stations in grey)

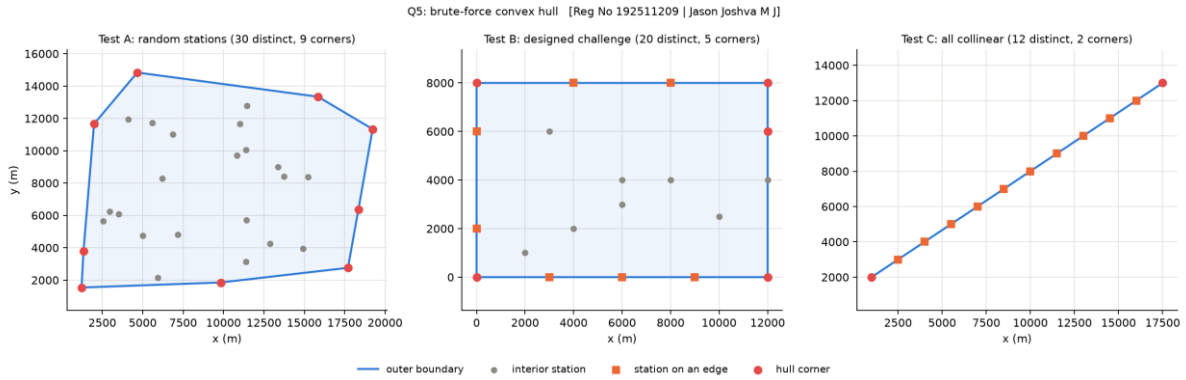


Figure 25: Terminal output of q5_convex_hull.py — tests A, B and C, and the naive rule on test B

```
================================================================================
 DAA Assignment 2 (CO2)  |  Q5 Brute-force convex hull - wildlife reserve boundary
 Reg No: 192511209  |  Name: Jason Joshva M J  |  Seed: 1209 (last 4 digits of reg no)
 Run at: 2026-10-16 21:20:19 (system local time)  |  Python 3.13.14
================================================================================

--- Test configurations ------------------------------------------------------

  Test A: 30 random stations (boundary and interior points)
    stations 30 (distinct 30), pairs examined 435, orientation tests 12,180, accepted edges 9
    hull corners (9, counter-clockwise): [(9825, 1851), (17703, 2757), (18350, 6372), (19229, 11337), (15851, 13339), (4657, 14835), (1984, 11655), (1330, 3782),
(1212, 1538)]
    stations on the boundary: 9  (corners + stations lying on an edge)
    interior stations: 21   check against monotone chain: PASS

  Test B (designed challenge): rectangle with stations on the fence, duplicates, an interior collinear line, one station 1 m outside
    stations 22 (distinct 20), pairs examined 190, orientation tests 3,420, accepted edges 5
    hull corners (5, counter-clockwise): [(12000, 0), (12001, 6000), (12000, 8000), (0, 8000), (0, 0)]
    stations on the boundary: 12  (corners + stations lying on an edge)
    interior stations: 8   check against monotone chain: PASS

  Test C (degenerate): all 12 stations along a straight river bank
    stations 12 (distinct 12), pairs examined 66, orientation tests 660, accepted edges 1
    hull corners (2, counter-clockwise): [(1000, 2000), (17500, 13000)]
    stations on the boundary: 12  (corners + stations lying on an edge)
    interior stations: 0   check against monotone chain: PASS

  Same test B with the naive rule (collinear points ignored): 24 edges accepted instead of 5 -> boundary FAILS: accepted edges do not form a single closed boundar
y (vertex degrees [2, 3, 4, 5, 6, 7])
```

Why test B mattered. With the simple rule "all other points on one side" and collinear points ignored, test B accepted **24 edges instead of 5**: every pair of fence stations on the same side of the rectangle passes that test. Some stations then belonged to as many as 7 accepted edges, so no single closed boundary could be traced. The challenge configuration therefore exposed a real flaw in the naive reading of the principle, and the "between" rule fixed it.

6.5 Computational Analysis

There are n(n − 1)/2 candidate pairs, and each is tested against the other n − 2 stations, so the full algorithm performs exactly:

$$T(n) = \frac{n(n-1)}{2}(n-2) = \frac{n(n-1)(n-2)}{2} \in \Theta(n^3)$$

Table 6.3: Growth with the Number of Stations (from results/q5_growth.csv)

n	Pairs	Theory n(n−1)(n−2)/2	Tests (full)	Tests (early exit)	Corners	Time (ms)
10	45	360	360	214	6	0.24
20	190	3,420	3,420	1,273	8	1.27
40	780	29,640	29,640	9,197	11	11.61
80	3,160	246,480	246,480	59,686	12	73.10
160	12,720	2,009,760	2,009,760	441,243	18	853.64
320	51,040	16,230,720	16,230,720	3,356,617	14	5,077.43

The measured tests equal the formula at every size. Each doubling of n multiplied the tests by 9.50, 8.67, 8.32, 8.15 and 8.08, approaching the factor 8 expected of $\Theta(n^3)$ (the small-n ratios are larger because (n − 1)(n − 2) is far from n^2 when n is small). At 320 stations the program already needed 16.2 million orientation tests and about 5 seconds.

Early exit. A pair can be rejected as soon as one station is found on each side. With this change the tests fell to 3,356,617 at n = 320, about 21% of the full count. This is a useful constant-factor saving, but the early-exit counts still grew by ×5.9 to ×7.6 per doubling, approaching 8 (Figure 26), so the algorithm remains cubic. Most pairs are not edges (only 14 of 51,040 at n = 320), yet every pair has to be examined.

Figure 26: Orientation tests against n (log–log), full test and early exit, with the n(n − 1)(n − 2)/2 curve

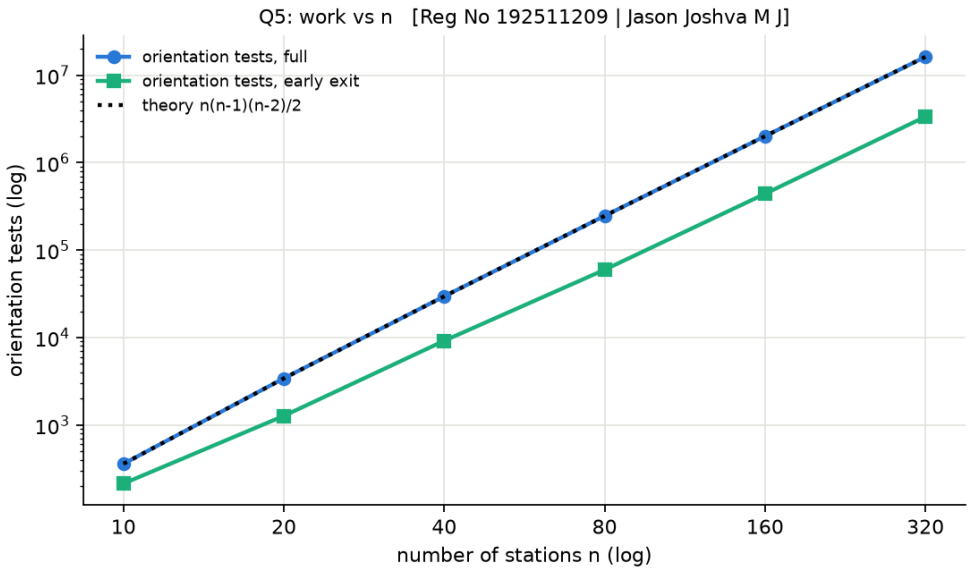


Figure 27: Terminal output — edge cases, growth and the doubling-ratio check

```
--- Edge cases ------------------------------------------------------------------------
  case                    input  distinct  pairs  tests  corners  hull                          check
  ----------------------  -----  --------  -----  -----  -------  ----------------------------  -----
  1 station                   1         1      0      0        1  [(500, 500)]                  PASS
  2 stations                  2         2      1      0        2  [(0, 0), (900, 300)]          PASS
  3 stations, triangle        3         3      3      3        3  [(900, 0), (400, 700), (0, 0)] PASS
  3 stations, collinear       3         3      3      3        2  [(0, 0), (1000, 1000)]        PASS
  4 stations, one inside      4         4      6     12        3  [(900, 0), (400, 700), (0, 0)] PASS
  5 copies of one station     5         1      0      0        1  [(7, 7)]                      PASS
  [saved results\q5_tests.csv]

--- Growth: random stations, full test vs early exit ----------------------------------
  n    distinct  theory pairs  pairs   theory tests n(n-1)(n-2)/2  tests (full)  tests (early exit)  corners h  time ms  check
  ---  --------  ------------  ------  --------------------------  ------------  ------------------  ---------  -------  -----
   10        10            45      45                         360           360                 214          6   0.2388  PASS
   20        20           190     190                       3,420         3,420               1,273          8    1.271  PASS
   40        40           780     780                      29,640        29,640               9,197         11   11.607  PASS
   80        80         3,160   3,160                     246,480       246,480              59,686         12   73.101  PASS
  160       160        12,720  12,720                   2,009,760     2,009,760             441,243         18  853.643  PASS
  320       320        51,040  51,040                  16,230,720    16,230,720           3,356,617         14  5,077.4  PASS
  [saved results\q5_growth.csv]

  ratio check when n doubles (theory: tests about x8):
    n   10 ->   20: tests x9.50, time x5.32
    n   20 ->   40: tests x8.67, time x9.13
    n   40 ->   80: tests x8.32, time x6.30
    n   80 ->  160: tests x8.15, time x11.68
    n  160 ->  320: tests x8.08, time x5.95
  [graph results\graphs\q5_hulls.png]
  [graph results\graphs\q5_growth.png]

  [q5 finished in 8.6 s]
```

6.6 Correctness Inquiry — Why an Accepted Edge Belongs to the Outer Boundary

The argument below relies only on geometry, not on the program's output.

1. **The accepted condition puts every station in one closed half-plane.** If no station is strictly on the left of the line L through P_i and P_j (or none is on the right), all stations lie in the closed half-plane H bounded by L.

2. **The hull lies in that half-plane.** A half-plane is convex, and the convex hull is the smallest convex set that contains every station. Since H is convex and contains every station, the hull is inside H. So no part of the hull crosses L.
3. **The segment between the two stations lies on the hull's boundary.** P_i and P_j are stations, so they are in the hull, and by convexity so is the whole segment between them. The segment lies on L, the edge of H, and the hull does not extend past L. Every point of the segment therefore has hull points on one side only, so it is on the outer boundary. L is a supporting line that touches the hull along P_iP_j.
4. **The "between" rule makes it a complete edge.** If a collinear station lay beyond P_j, the hull would touch L along a longer segment and P_iP_j would be only part of it. Requiring every collinear station to lie between P_i and P_j makes P_iP_j the entire contact between the hull and L, so its endpoints are true corners.
5. **No edge is missed.** Conversely, each edge of the hull lies on a supporting line with every station on one side, and its endpoints are the extreme stations on that line. So every true edge passes the test, and the accepted edges are exactly the hull's edges.

Because the orientation test is computed exactly in integers, steps 1 and 4 are decided without rounding error. That is why the station 1 m outside a 12 km fence was classified correctly in test B.

7. Q6 — EXHAUSTIVE SEARCH AND COMBINATORIAL GROWTH

7.1 Designing the Exhaustive Search

A helicopter carrying emergency supplies has a strict payload limit C. Each package has an ID, a weight and a humanitarian value. The goal is a set of packages whose total weight does not exceed C and whose total value is as large as possible. I designed the exhaustive search from five decisions:

Table 7.1: Design of the Exhaustive Search

Decision	Choice and reason
Representation of a candidate	An n-bit number (a mask). Bit i = 1 means package i is loaded. Every subset of the n packages corresponds to exactly one number from 0 to $2^n - 1$, so the representation covers the whole solution space without gaps or repeats.
Generation of candidates	Count mask = 0, 1, 2, …, $2^n - 1$. This lists every subset exactly once and needs no extra memory.
Feasibility check	Add up the weights of the packages whose bit is set; the load is feasible if the total weight ≤ C. Equality is allowed ("does not exceed").
Evaluation	Add up the values of the same packages.
Retaining the best	Keep the feasible load with the highest value seen so far; replace it only by a strictly better one.

```
EXHAUSTIVE-LOAD(packages[0 .. n-1], C)
best = -1; bestMask = 0
for mask = 0 .. 2^n - 1                    # every candidate load
    w = 0; v = 0
    for i = 0 .. n-1                         # evaluate it from scratch
        itemChecks += 1
        if bit i of mask is 1: w += weight[i]; v += value[i]
    if w <= C and v > best                   # feasible and better
        best = v; bestMask = mask
return best, the packages whose bits are set in bestMask
```

7.2 Correctness Check on a Small Instance

I built a 4-package instance whose $2^4 = 16$ subsets can be listed by hand. The payload limit is 50 kg. The instance is deliberately designed so that the best load fills the limit exactly (testing the "≤" condition) and so that the tempting shortcut "take the best value per kg first" fails.

Table 7.2: The Four Packages

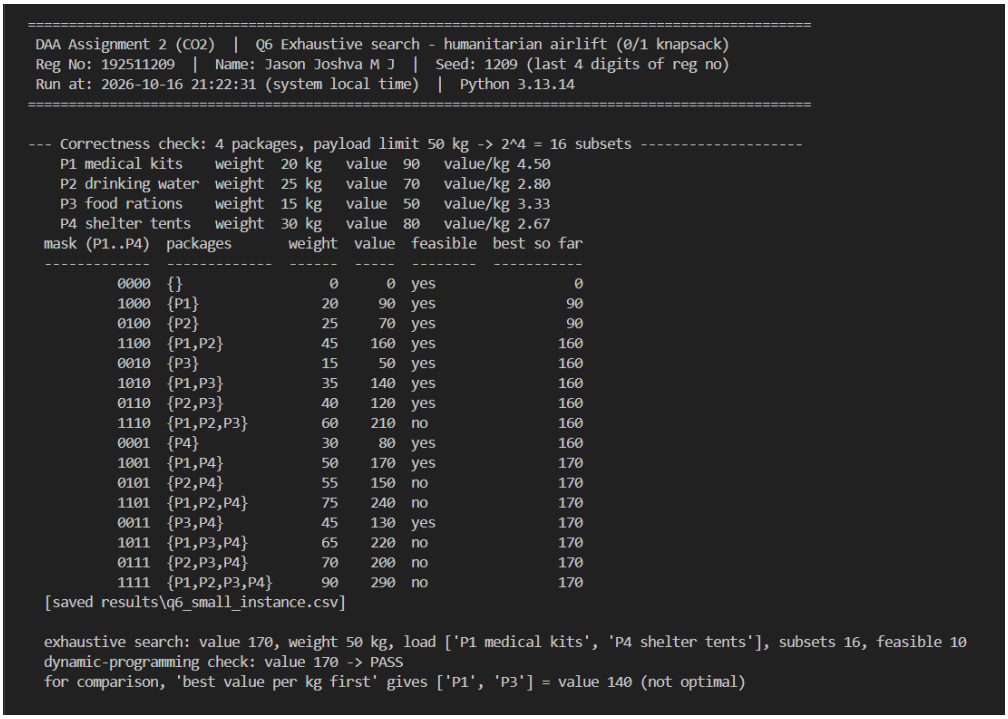
Package	Weight (kg)	Value	Value per kg
P1 medical kits	20	90	4.50
P2 drinking water	25	70	2.80
P3 food rations	15	50	3.33
P4 shelter tents	30	80	2.67

Table 7.3: All 16 Candidate Loads (from results/q6_small_instance.csv)

Mask (P1..P4)	Packages	Weight	Value	Feasible	Best so far
0000	{ }	0	0	yes	0
1000	{P1}	20	90	yes	90
0100	{P2}	25	70	yes	90
1100	{P1, P2}	45	160	yes	160
0010	{P3}	15	50	yes	160
1010	{P1, P3}	35	140	yes	160
0110	{P2, P3}	40	120	yes	160
1110	{P1, P2, P3}	60	210	no	160
0001	{P4}	30	80	yes	160
1001	**{P1, P4}**	**50**	**170**	**yes**	**170**
0101	{P2, P4}	55	150	no	170
1101	{P1, P2, P4}	75	240	no	170
0011	{P3, P4}	45	130	yes	170
1011	{P1, P3, P4}	65	220	no	170
0111	{P2, P3, P4}	70	200	no	170
1111	{P1, P2, P3, P4}	90	290	no	170

By inspection, 10 of the 16 loads are feasible and the best is {P1, P4}: 50 kg and value 170, exactly at the limit. The program found the same optimum after generating 16 subsets, and an independent dynamic-programming check also returned 170 (PASS). The "best value per kg first" shortcut would choose P1 and then P3 (35 kg), after which neither P2 nor P4 fits, giving only 140. This shows why the exhaustive search is needed as a reference: a plausible shortcut misses the optimum even on four packages.

Figure 28: Terminal output of q6_knapsack_exhaustive.py — the 16 subsets, the optimum, the DP check and the greedy comparison



7.3 Prediction of Solution-Space Growth

Each package is either loaded or not, so n packages give $2 \times 2 \times \cdots \times 2 = 2^n$ candidate loads. Every candidate is evaluated with n item checks, so the work grows as $n \cdot 2^n$:

$$|S(n)| = 2^n, \qquad W(n) = n \cdot 2^n, \qquad \frac{|S(n+1)|}{|S(n)|} = 2, \qquad \frac{W(n+2)}{W(n)} = \frac{4(n+2)}{n}$$

Prediction before the experiment: every extra package doubles the number of candidates, so going from n to n + 2 multiplies them by 4, and adding 10 packages multiplies them by $2^{10} = 1{,}024$. The item checks grow slightly faster (×5.00 from 8 to 10 packages, ×4.44 from 18 to 20), because each candidate also gets longer. Time should follow the item checks.

7.4 Growth Experiment

All instances come from one manifest of packages generated from my seed (weights 5–60 kg, values 10–100). An instance of size n uses the first n packages, so a larger instance is literally the same manifest with more packages added. The payload limit is 40% of the total weight of the available packages, which keeps every instance non-trivial (the answer is neither "take everything" nor "take nothing").

Table 7.4: Growth with the Number of Packages (from results/q6_growth.csv)

n	Limit (kg)	Subsets 2^n	Feasible	Feasible (%)	Item checks	Best value	Time (ms)	DP check
8	135	256	79	30.9	2,048	270	0.40	PASS
10	171	1,024	281	27.4	10,240	366	1.48	PASS
12	204	4,096	1,031	25.2	49,152	434	6.55	PASS
14	208	16,384	4,062	24.8	229,376	529	30.92	PASS
16	241	65,536	15,346	23.4	1,048,576	660	141.32	PASS
18	278	262,144	57,396	21.9	4,718,592	737	1,284.30	PASS
20	312	1,048,576	213,967	20.4	20,971,520	801	3,304.37	PASS

Validation of the prediction. The subsets generated were exactly 2^n at every size and grew ×4.00 at every step of +2 packages. The item checks grew ×5.00, ×4.80, ×4.67, ×4.57, ×4.50 and ×4.44 — exactly $4(n + 2)/n$. The optimum agreed with dynamic programming every time. Time followed the same trend overall (0.40 ms to 3,304 ms, ×8,350 for ×10,240 item checks), with one irregular step discussed in Section 7.6.

Figure 29: Left — candidate and feasible subsets against n (log scale); right — time against n with the 1 s deadline

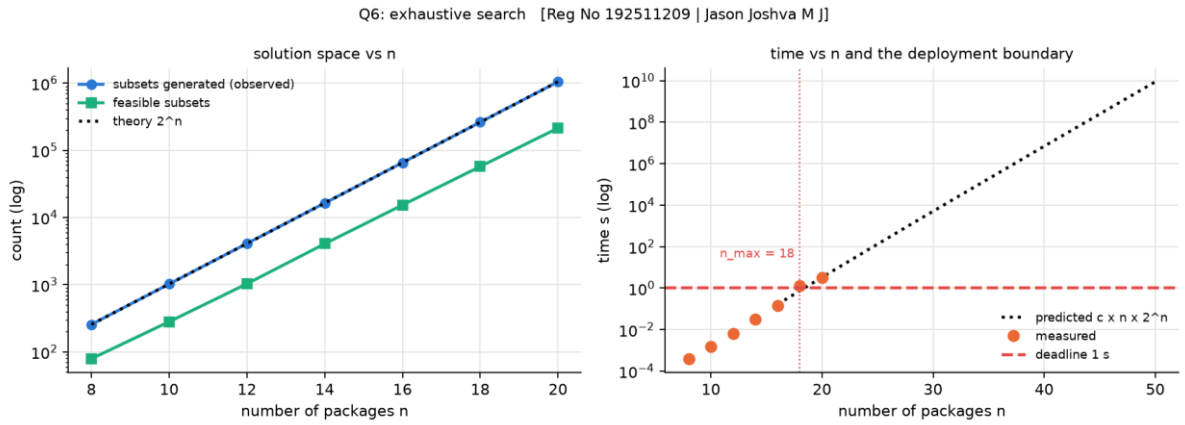


Figure 30: Terminal output — prediction table, growth experiment and the ratio check

```
--- Prediction before running: candidates = 2^n, item checks = n x 2^n ------------------------
  n   2^n subsets  n x 2^n item checks  vs previous size
  --  -----------  -------------------  ----------------
   8          256                2,048  -
  10        1,024               10,240  x4 subsets
  12        4,096               49,152  x4 subsets
  14       16,384              229,376  x4 subsets
  16       65,536            1,048,576  x4 subsets
  18      262,144            4,718,592  x4 subsets
  20    1,048,576           20,971,520  x4 subsets

--- Experiment: growth with the number of packages (same manifest, extra packages added) ------
  n   limit kg  2^n        subsets    feasible  feasible %  item checks  additions   best value  weight  packages  time ms  ns/subset  DP check
  --  --------  ---------  ---------  --------  ----------  -----------  ----------  ----------  ------  --------  -------  ---------  --------
   8       135        256        256        79      30.859        2,048       1,024         270     122         3   0.3956    1,545.4  PASS
  10       171      1,024      1,024       281      27.441       10,240       5,120         366     170         4    1.478    1,443.2  PASS
  12       204      4,096      4,096     1,031      25.171       49,152      24,576         434     204         5    6.550    1,599.0  PASS
  14       208     16,384     16,384     4,062      24.792      229,376     114,688         529     198         7   30.918    1,887.1  PASS
  16       241     65,536     65,536    15,346      23.416    1,048,576     524,288         660     241         8  141.324    2,156.4  PASS
  18       278    262,144    262,144    57,396      21.895    4,718,592   2,359,296         737     278         9  1,284.3    4,899.2  PASS
  20       312  1,048,576  1,048,576   213,967      20.405   20,971,520  10,485,760         801     310        10  3,304.4    3,151.3  PASS
  [saved results\q6_growth.csv]

  ratio check (n + 2 -> theory x4 subsets):
    n  8 -> 10: subsets x4.00, item checks x5.00, time x3.74
    n 10 -> 12: subsets x4.00, item checks x4.80, time x4.43
    n 12 -> 14: subsets x4.00, item checks x4.67, time x4.72
    n 14 -> 16: subsets x4.00, item checks x4.57, time x4.57
    n 16 -> 18: subsets x4.00, item checks x4.50, time x9.09
    n 18 -> 20: subsets x4.00, item checks x4.44, time x2.57
```

7.5 Inquiry — Why a Few Extra Packages Cause a Dramatic Increase

The solution space is the set of all subsets of the packages. When one package is added, every old candidate appears twice in the new space — once without the new package and once with it — so the space doubles no matter how light or unimportant the package is. The growth is multiplicative, not additive: in Table 7.4, going from 18 to 20 packages added only two packages but created 786,432 new candidates, three times more than all the candidates for 18 packages. Ten more packages (20 → 30) would multiply the candidates by 1,024 and, at the measured speed, turn a 3.3-second plan into about 1.4 hours. Because the exhaustive search must look at every candidate, its effort grows with the size of the solution space, which is exponential in n.

7.6 Deployment Decision

Response time: 1 second. The planner is used interactively at the loading pad and is re-run every time the manifest changes (a package arrives, is damaged or becomes urgent). A response within about one second keeps the person's flow of work uninterrupted, while longer waits break concentration and hold up the loading crew. I therefore chose 1 s as the fixed deadline.

From the largest run (n = 20) the cost was 157.6 ns per item check, so the time for n packages can be predicted as:

$$t(n) \approx 157.6 \text{ ns} \times n \times 2^n \leq 1 \text{ s}$$

Table 7.5: Predicted and Measured Time (from results/q6_deployment.csv)

n	Subsets	Predicted	Measured	Within 1 s?
16	65,536	0.165 s	0.141 s	yes
17	131,072	0.351 s	—	yes
18	262,144	0.743 s	1.284 s	model yes, measured no
19	524,288	1.57 s	—	no
20	1,048,576	3.30 s	3.304 s	no
22	4,194,304	14.5 s	—	no
25	33,554,432	2.2 minutes	—	no
30	1,073,741,824	1.4 hours	—	no
40	1,099,511,627,776	80.2 days	—	no

Figure 31: Terminal output — deployment decision

```
--- Deployment decision: loading plan must be ready within 1 s -------------------------------
  measured cost per item check (n = 20): 157.6 ns  ->  time(n) ~ 157.6 ns x n x 2^n
  n   subsets               predicted s      measured s  within 1 s?  predicted (readable)
  --  --------------------  ---------------  ----------  -----------  --------------------
  16                65,536           0.1652       0.141  yes          165 ms
  17               131,072           0.3511  -           yes          351 ms
  18               262,144           0.7435       1.284  yes          743 ms
  19               524,288            1.570  -           NO           1.57 s
  20             1,048,576            3.304       3.304  NO           3.30 s
  21             2,097,152            6.939  -           NO           6.94 s
  22             4,194,304           14.539  -           NO           14.54 s
  23             8,388,608           30.400  -           NO           30.40 s
  24            16,777,216           63.444  -           NO           1.1 minutes
  25            33,554,432          132.175  -           NO           2.2 minutes
  26            67,108,864          274.924  -           NO           4.6 minutes
  27           134,217,728          570.995  -           NO           9.5 minutes
  28           268,435,456          1,184.3  -           NO           19.7 minutes
  29           536,870,912          2,453.2  -           NO           40.9 minutes
  30         1,073,741,824          5,075.5  -           NO           1.4 hours
  32         4,294,967,296         21,655.5  -           NO           6.0 hours
  40     1,099,511,627,776      6,929,764.1  -           NO           80.2 days
  50 1,125,899,906,842,624 8,870,097,993.1  -           NO           281.1 years
  [saved results\q6_deployment.csv]

  largest n within the deadline: n = 18 packages
  [graph results\graphs\q6_growth.png]

  [q6 finished in 6.8 s]
```

The model gives n = 18 as the largest size within 1 s (0.743 s), and the program prints this. However, my measured run at n = 18 took **1.284 s**, which is over the deadline. The cost per item check at n = 18 was 272 ns, against 135 ns at n = 16 and 158 ns at n = 20; when Q7 re-ran the same n = 18 instance it took 0.969 s (205 ns per check). I did not find the cause — it may be the laptop's power management or other programs — but it shows that timings on this machine vary by up to a factor of two.

Decision: I would deploy the exhaustive search for at most 17 packages. At the rate measured at n = 20 this takes 0.35 s, and even at the slowest rate seen (272 ns per check) it

takes 0.61 s, still within the deadline. At 18 packages the deadline is met only on a good run, and at 19 packages it is missed even by the model. So the exhaustive planner covers only very small airlifts, which is the limitation addressed in Q7.

8. Q7 — DIAGNOSE, REDESIGN AND VALIDATE

8.1 Choice of the Solution to Redesign

Three of my solutions show a growing limitation: closest pair ($\Theta(n^2)$), impractical beyond about 1,000–1,400 drones), convex hull ($\Theta(n^3)$), 16.2 million tests for 320 stations) and the exhaustive airlift planner ($\Theta(n \cdot 2^n)$), deployable only up to 17 packages). I chose the **airlift planner from Q6**, because its growth is exponential — each extra package doubles the work — and its deployment boundary is the most severe: a realistic helicopter load can easily contain more than 17 packages.

8.2 A. Diagnosis of the Computational Bottleneck

I re-ran the original algorithm on the same instances as Q6 with extra counters (Table 8.1).

Table 8.1: Where the Original Spends Its Effort (from results/q7_diagnosis.csv)

n	Subsets	Feasible	Infeasible %	Item checks	Additions	Additions per subset	Repeat factor
8	256	79	69.1	2,048	1,024	4.0	4.0
10	1,024	281	72.6	10,240	5,120	5.0	5.0
12	4,096	1,031	74.8	49,152	24,576	6.0	6.0
14	16,384	4,062	75.2	229,376	114,688	7.0	7.0
16	65,536	15,346	76.6	1,048,576	524,288	8.0	8.0
18	262,144	57,396	78.1	4,718,592	2,359,296	9.0	9.0
20	1,048,576	213,967	79.6	20,971,520	10,485,760	10.0	10.0

Repeat factor = additions performed ÷ the $2^n - 1$ additions that would be needed if each subset reused the sums of a subset that differs from it by one package.

- **Where most effort is spent:** in the inner loop that evaluates a subset — n item checks per subset, 20,971,520 at n = 20. Everything else (one feasibility comparison and one value comparison per subset) is 20 times smaller.
- **Which operation grows significantly:** the item checks, $n \cdot 2^n$. They grew ×4.44 to ×5.00 for every two packages added (Table 7.4).
- **Whether work is repeated:** yes, in two ways. (1) Each subset is summed from scratch, although it differs from an already-evaluated subset by a single package: n/2 additions per subset instead of one, a repeat factor of 10 at n = 20. (2) **79.6% of all generated loads at n = 20 are infeasible**, and their evaluation is wasted. Worse, most of them extend a load that was already over the limit — once a partial load is too heavy, every

larger load containing it is too heavy as well, but the original generates and checks each one anyway.

- **The characteristic that causes the limitation:** the original is **generate first, test later**. It lists all 2^n masks blindly, without using the payload limit or the package values while generating, and only judges a load after it is complete. Nothing it learns about one candidate (too heavy, or obviously worse than the best so far) is used to avoid others.

8.3 B. Design Objective

The redesign must keep the guarantee of the original — the true optimum — while removing the diagnosed waste. I set three measurable objectives:

Table 8.2: Measurable Objectives

	Objective	**How it is measured**
O1	Return the same optimum value as the original on every instance (and as dynamic programming)	Best value compared on n = 8–20 (and DP up to n = 80)
O2	At n = 20, perform at least 90% fewer elementary operations than the original	Original: item checks. Redesign: nodes visited + bound steps
O3	Plan a 40-package airlift within the 1 s deadline (the original's limit is 17)	Measured time at n = 40

The design targets map directly to the diagnosis: **reuse previously computed information** (the sums of a partial load), **reduce the number of candidate solutions** (never generate infeasible loads), and **avoid examining every candidate** (skip loads that cannot beat the best one).

8.4 C. Revised Computational Strategy

Instead of listing complete masks, the redesign builds each load one decision at a time: for package i, either load it or leave it. This forms a binary decision tree whose leaves are the 2^n complete loads. I introduced three changes in separate steps, so that the effect of each idea can be measured on its own:

Table 8.3: What Each Step Changes

Version	**What changes**	**Property it exploits**	**Operations avoided / new cost**
V1 reuse sums	Depth-first search over the decision tree, carrying the running weight and value	A child load differs from its parent by one package, so its sums are the parent's sums plus one addition	Avoids re-summing each subset (n checks → 1 step per node). Cost: $2^{n+1} - 1$ nodes and a recursion stack of depth n

Version	What changes	Property it exploits	Operations avoided / new cost
V2 + capacity cut	Take the "load it" branch only if the package still fits	Weights are positive, so an overweight partial load can never become feasible again	Never generates an infeasible load (79.6% at n = 20). Cost: one comparison per branch
V3 + sort and bound	Sort packages by value per kg once; at each node compute an optimistic bound and skip the subtree if the bound cannot beat the best load found so far	Values are additive, and filling the remaining payload greedily (allowing a fraction of one package) can never be beaten by a real load	Skips whole subtrees of hopeless loads. Cost: sorting O(n log n) once, plus up to n − i bound steps at each node

The bound. With packages sorted by value per kg, the bound at a partial load (weight w, value v, next package i) adds the following packages while they fit, and then a fraction of the first one that does not fit:

$$U(i, w, v) = v + \sum_{j=i}^{k-1} v_j + v_k \cdot \frac{C - w - \sum_{j=i}^{k-1} w_j}{w_k}$$

Every real completion of the partial load is also a valid fractional filling, and the greedy fractional filling in value-per-kg order is the best possible fractional filling. So U is never smaller than the value of any real completion. If U ≤ best, no load below this node can beat the best load already found, and skipping it cannot lose the optimum. This is why V3 is still exact (objective O1).

```
PLAN-LOAD(packages, C)
items = packages sorted by value/weight, largest first     # preprocessing
best = 0; bestLoad = { }
VISIT(0, 0, 0, { })
return best, bestLoad

VISIT(i, w, v, load)               # w, v = weight and value of the partial load
nodes += 1
if i == n                           # a complete candidate load
    if v > best: best = v; bestLoad = load
    return
if BOUND(i, w, v) <= best: return   # V3: nothing below can beat best
if w + items[i].w <= C              # V2: only feasible branches
    VISIT(i+1, w + items[i].w, v + items[i].v, load + {i})   # V1: one addition
VISIT(i+1, w, v, load)              # leave package i

BOUND(i, w, v)                      # optimistic value of any completion
room = C - w; b = v
for j = i .. n-1
    if items[j].w <= room: room -= items[j].w; b += items[j].v
    else: return b + items[j].v * room / items[j].w        # a fraction of j
return b
```

Preprocessing and memory. V3 sorts the packages once (O(n log n)). The search needs a recursion stack of depth n + 1, the sorted copy of the packages and the current load — O(n) extra memory, against O(1) for the original's counter. **New trade-offs:** each node now costs more (a function call and, in V3, a bound computation); the running time depends on the data

rather than being fixed at $n \cdot 2^n$; in the worst case the tree is still exponential; and the code is harder to verify, which is why every run is checked against the original and against dynamic programming.

8.5 D. Prediction Before Implementing

Before running the redesign, the program printed these predictions for n = 20 (Figure 32):

Table 8.4: Predictions for n = 20

Version	Prediction	Reason
V1	Same 2^{20} = 1,048,576 candidates; $2^{21} - 1$ = 2,097,151 nodes instead of 20,971,520 item checks — about n/2 = 10× fewer operations	One step per node instead of n checks per subset; a full binary tree of depth n has $2^{n+1} - 1$ nodes
V2	Candidates drop to exactly the number of feasible loads, 213,967 (20.4%)	Every feasible load is still reached, and no infeasible one is
V3	Candidates far below the feasible count, but the gain depends on the data; weak when all packages have a similar value per kg	The bound is tight when values per kg differ, and loose when they are almost equal

The performance measures in which I expected improvement were the **elementary operations** and the **candidate loads evaluated** (primary), with execution time as supporting evidence.

Figure 32: Terminal output of q7_redesign.py — A (diagnosis), B (objectives) and D (predictions)

```
================================================================================
 DAA Assignment 2 (CO2)  |  Q7 Diagnose, redesign and validate - airlift planner (from Q6)
 Reg No: 192511209  |  Name: Jason Joshva M J  |  Seed: 1209 (last 4 digits of reg no)
 Run at: 2026-10-16 21:24:58 (system local time)  |  Python 3.13.14
================================================================================

--- A. Diagnosis: where the original exhaustive search spends its effort ----------
  n   subsets    feasible  infeasible  infeasible %  item checks  additions   additions/subset  additions if reused  repeat factor
  --  ---------  --------  ----------  ------------  -----------  ----------  ----------------  -------------------  -------------
   8        256        79         177        69.141        2,048       1,024             4.000                  255          4.016
  10      1,024       281         743        72.559       10,240       5,120             5.000                1,023          5.005
  12      4,096     1,031       3,065        74.829       49,152      24,576             6.000                4,095          6.001
  14     16,384     4,062      12,322        75.208      229,376     114,688             7.000               16,383          7.000
  16     65,536    15,346      50,190        76.584    1,048,576     524,288             8.000               65,535          8.000
  18    262,144    57,396     204,748        78.105    4,718,592   2,359,296             9.000              262,143          9.000
  20  1,048,576   213,967     834,609        79.595   20,971,520  10,485,760            10.000            1,048,575         10.000
  [saved results\q7_diagnosis.csv]

  at n = 20: 79.6% of all generated loads break the payload limit (all their work is wasted),
  every subset is re-summed from scratch (10.0 additions each, x10.0 more than needed),
  and the limit is only checked after the subset is complete (generate first, test later).

--- B. Design objective (measurable) ----------------------------------------------
  O1  return the same optimum as the original (and as an independent DP check) on every instance
  O2  at n = 20: at least 90% fewer elementary operations than the original
  O3  plan a 40-package airlift within the 1 s deadline (original limit: about 18)

--- D. Prediction written before running the redesign -----------------------------
  V1: same 2^n = 1,048,576 candidates; nodes = 2^(n+1)-1 = 2,097,151 instead of n x 2^n = 20,971,520 item checks -> about n/2 = 10x fewer operations
  V2: candidates (leaves) drop to exactly the feasible loads = 213,967 (20.4%)
  V3: candidates drop far below the feasible count, but the gain depends on the data
      (expected to be weak when all packages have a similar value per kg)
```

8.6 E. Implementation and Validation

All four versions were run on exactly the same instances as Q6 (Table 8.5). "Candidates" are complete loads evaluated (subsets for V0, leaves for V1–V3); "total ops" are item checks for V0 and nodes + bound steps for the redesign.

Table 8.5: Original Against Redesign on the Q6 Instances (from results/q7_comparison.csv)

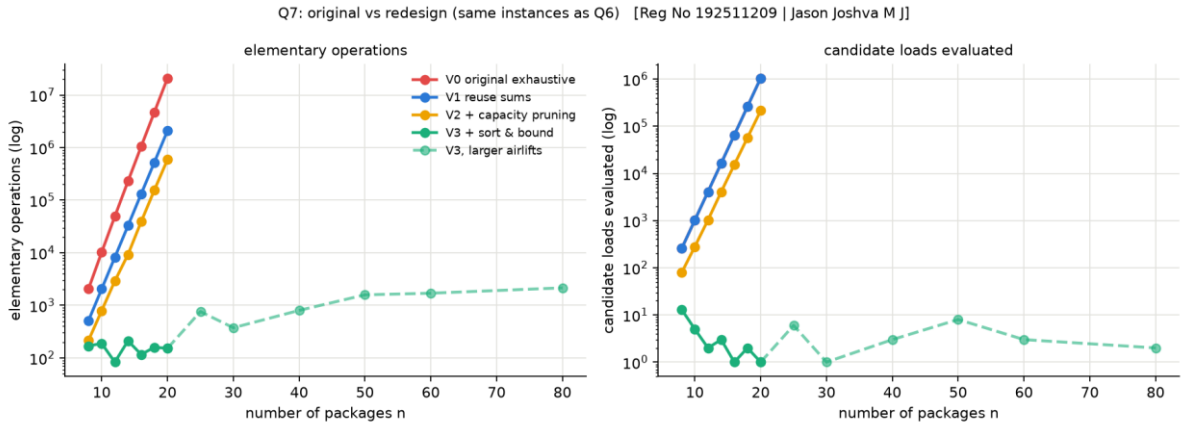
n	Version	Candidates	Nodes	Bound steps	Total ops	Best value	Time (ms)	Ops gain
8	V0 original	256	256	0	2,048	270	0.355	1×
8	V1 reuse sums	256	511	0	511	270	0.140	4.0×
8	V2 + capacity	79	215	0	215	270	0.051	9.5×
8	V3 + bound	13	74	95	169	270	0.036	12.1×
12	V0 original	4,096	4,096	0	49,152	434	6.139	1×
12	V1 reuse sums	4,096	8,191	0	8,191	434	1.906	6.0×
12	V2 + capacity	1,031	2,898	0	2,898	434	0.726	17.0×
12	V3 + bound	2	30	53	83	434	0.021	592×
16	V0 original	65,536	65,536	0	1,048,576	660	139.713	1×
16	V1 reuse sums	65,536	131,071	0	131,071	660	32.074	8.0×
16	V2 + capacity	15,346	38,901	0	38,901	660	9.743	27.0×
16	V3 + bound	1	25	91	116	660	0.026	9,039×
20	V0 original	1,048,576	1,048,576	0	20,971,520	801	3,057.177	1×
20	V1 reuse sums	1,048,576	2,097,151	0	2,097,151	801	470.375	10.0×
20	V2 + capacity	213,967	592,192	0	592,192	801	135.405	35.4×
20	V3 + bound	1	32	120	152	801	0.031	137,971×

The full table for n = 8, 10, 12, 14, 16, 18 and 20 (28 rows) is in results/q7_comparison.csv and in Figure 33. Every row reports the same optimum as the original (PASS).

Figure 33: Terminal output — the complete comparison of the original and the redesign on the Q6 instances

```
--- E. Original vs redesign on the same instances as Q6 ------------------------------------
  n  variant                candidates  nodes      item steps  bound steps  total ops   best value  same optimum  time ms  ops gain x  time gain x
 --  ---------------------  ----------  ---------  ----------  -----------  ----------  ----------  ------------  -------  ----------  -----------
  8  V0 original exhaustive         256        256       2,048            0       2,048         270  PASS           0.3551       1.000        1.000
  8  V1 reuse sums                  256        511         511            0         511         270  PASS             0.14       4.008        2.537
  8  V2 + capacity pruning           79        215         215            0         215         270  PASS          0.05111       9.526        6.948
  8  V3 + sort & bound               13         74          74           95         169         270  PASS          0.03627      12.118        9.790
 10  V0 original exhaustive      1,024      1,024      10,240            0      10,240         366  PASS            1.576       1.000        1.000
 10  V1 reuse sums               1,024      2,047       2,047            0       2,047         366  PASS           0.4179       5.002        3.771
 10  V2 + capacity pruning         281        791         791            0         791         366  PASS           0.2132      12.946        7.391
 10  V3 + sort & bound               5         75          75          113         188         366  PASS           0.0408      54.468       38.627
 12  V0 original exhaustive      4,096      4,096      49,152            0      49,152         434  PASS            6.139       1.000        1.000
 12  V1 reuse sums               4,096      8,191       8,191            0       8,191         434  PASS            1.906       6.001        3.221
 12  V2 + capacity pruning       1,031      2,898       2,898            0       2,898         434  PASS           0.7259      16.961        8.457
 12  V3 + sort & bound               2         30          30           53          83         434  PASS          0.02091     592.193      293.570
 14  V0 original exhaustive     16,384     16,384     229,376            0     229,376         529  PASS           32.788       1.000        1.000
 14  V1 reuse sums              16,384     32,767      32,767            0      32,767         529  PASS            7.855       7.000        4.174
 14  V2 + capacity pruning       4,062      9,265       9,265            0       9,265         529  PASS            2.178      24.757       15.056
 14  V3 + sort & bound               3         74          74          137         211         529  PASS          0.05437   1,087.1        603.104
 16  V0 original exhaustive     65,536     65,536   1,048,576            0   1,048,576         660  PASS          139.713       1.000        1.000
 16  V1 reuse sums              65,536    131,071     131,071            0     131,071         660  PASS           32.074       8.000        4.356
 16  V2 + capacity pruning      15,346     38,901      38,901            0      38,901         660  PASS            9.743      26.955       14.340
 16  V3 + sort & bound               1         25          25           91         116         660  PASS           0.0256   9,039.4      5,458.1
 18  V0 original exhaustive    262,144    262,144   4,718,592            0   4,718,592         737  PASS          968.820       1.000        1.000
 18  V1 reuse sums             262,144    524,287     524,287            0     524,287         737  PASS          136.259       9.000        7.110
 18  V2 + capacity pruning      57,396    155,083     155,083            0     155,083         737  PASS           35.905      30.426       26.983
 18  V3 + sort & bound               2         42          42          117         159         737  PASS          0.03212  29,676.7     30,159.0
 20  V0 original exhaustive  1,048,576  1,048,576  20,971,520            0  20,971,520         801  PASS          3,057.2       1.000        1.000
 20  V1 reuse sums           1,048,576  2,097,151   2,097,151            0   2,097,151         801  PASS          470.375      10.000        6.499
 20  V2 + capacity pruning     213,967    592,192     592,192            0     592,192         801  PASS          135.405      35.413       22.578
 20  V3 + sort & bound               1         32          32          120         152         801  PASS          0.03125  137,970.5     97,830.6
 [saved results\q7_comparison.csv]
```

Figure 34: Elementary operations (left) and candidate loads evaluated (right) against n, log scale; the dashed line continues V3 to larger airlifts



Larger airlifts. Because V3 no longer depends on 2^n in practice, I extended the same manifest to 80 packages, checking every result with dynamic programming (Table 8.6).

Table 8.6: V3 on Larger Airlifts (from results/q7_large.csv)

n	Limit (kg)	Subsets 2^n	Candidates	Nodes	Total ops	Best value	DP check	Time (ms)
20	312	1,048,576	1	32	152	801	PASS	0.046
25	385	33,554,432	6	265	759	975	PASS	0.194
30	423	1,073,741,824	1	67	374	1,224	PASS	0.073
40	590	$\approx 1.1 \times 10^{12}$	3	174	800	1,552	PASS	0.158
50	720	$\approx 1.1 \times 10^{15}$	8	432	1,582	1,968	PASS	0.364
60	839	$\approx 1.2 \times 10^{18}$	3	312	1,698	2,303	PASS	0.374

n	Limit (kg)	Subsets 2^n	Candidates	Nodes	Total ops	Best value	DP check	Time (ms)
80	1,073	$\approx 1.2 \times 10^{24}$	2	213	2,149	3,040	PASS	0.332

At the original's measured speed (from its n = 20 run in Q7), the same 40-package problem would have needed about 74 days (Figure 35).

Limitation test. To test the prediction that V3 is weak when values per kg are similar, I generated five random instances per size of two kinds: uncorrelated (independent weights and values, like the manifest) and **strongly correlated** (value = weight + 10, so every package has almost the same value per kg). The search was stopped if it exceeded 2,000,000 nodes (about one second).

Table 8.7: V3 on Uncorrelated and Strongly Correlated Instances, 5 per Size (from results/q7_limitation.csv)

Instance class	n	Median nodes	Max nodes	Hit the cap	Max time (ms)
Uncorrelated	10	46	49	0	0.07
Uncorrelated	30	74	120	0	0.14
Uncorrelated	60	161	255	0	0.44
Strongly correlated	10	67	179	0	0.09
Strongly correlated	20	241	1,837	0	1.22
Strongly correlated	30	2,044	35,315	0	17.56
Strongly correlated	40	4,487	194,407	0	97.77
Strongly correlated	50	2,344	658,227	0	392.31
Strongly correlated	60	37,296	2,000,001	1 of 5	1,083.56

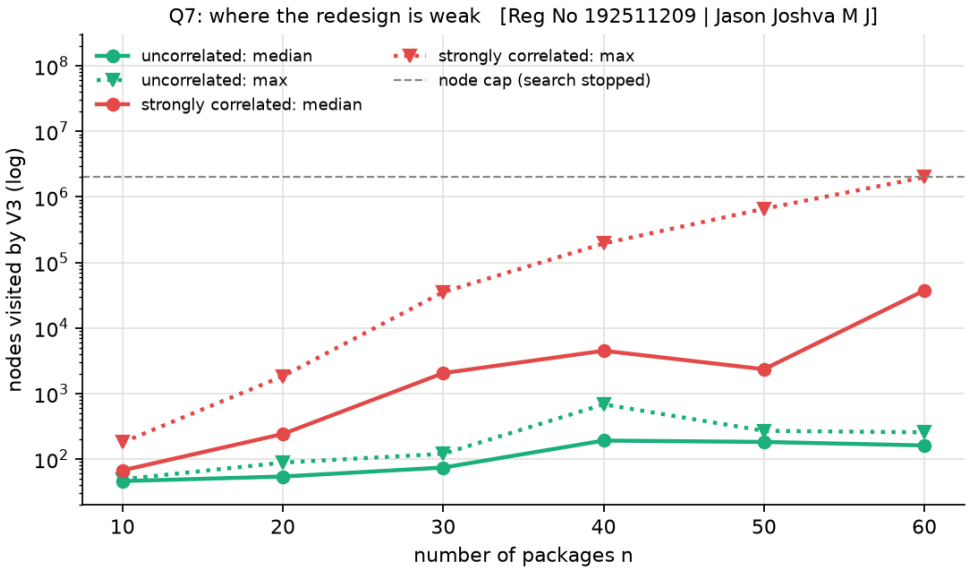
Figure 35: Terminal output — larger airlifts (E2), the limitation test (F) and the objective check

```
--- E2. Larger airlifts with V3 (same manifest, more packages) --------------------------------
  n   limit kg  2^n                              candidates  nodes  bound steps  total ops  best value  DP check  V3 time ms  original would need      within 1 s
  --  --------  -----------------------------------  ----------  -----  -----------  ---------  ----------  --------  ----------  ----------------------  ----------
  20       312                        1,048,576           1     32          120        152         801  PASS         0.04634  3.06 s                  yes
  25       385                       33,554,432           6    265          494        759         975  PASS          0.1944  2.0 minutes             yes
  30       423                    1,073,741,824           1     67          307        374       1,224  PASS         0.07282  1.3 hours               yes
  40       590                1,099,511,627,776           3    174          626        800       1,552  PASS          0.1578  74.2 days               yes
  50       720            1,125,899,906,842,624           8    432        1,150      1,582       1,968  PASS          0.3643  260.0 years             yes
  60       839        1,152,921,504,606,846,976           3    312        1,386      1,698       2,303  PASS          0.3744  319,549.2 years         yes
  80     1,073  1,208,925,819,614,629,174,706,176           2    213        1,936      2,149       3,040  PASS          0.3318  446,762,110,097.0 years yes
  [saved results\q7_large.csv]

--- F. Limitation test: 5 instances per size, uncorrelated vs strongly correlated (node cap 2,000,000) ----
  instance class       n   median nodes  max nodes  hit cap  max time ms  DP check
  -----------------  --  ------------  ---------  -------  -----------  --------
  uncorrelated         10            46         49        0       0.0689  PASS
  uncorrelated         20            54         88        0       0.0649  PASS
  uncorrelated         30            74        120        0       0.1362  PASS
  uncorrelated         40           191        682        0       0.3921  PASS
  uncorrelated         50           182        269        0       0.5604  PASS
  uncorrelated         60           161        255        0       0.4353  PASS
  strongly correlated  10            67        179        0       0.0938  PASS
  strongly correlated  20           241      1,837        0        1.221  PASS
  strongly correlated  30         2,044     35,315        0       17.564  PASS
  strongly correlated  40         4,487    194,407        0       97.773  PASS
  strongly correlated  50         2,344    658,227        0       392.309  PASS
  strongly correlated  60        37,296  2,000,001        1       1,083.6  PASS
  (a run that hits the node cap is stopped without an answer; the DP check covers the finished runs)
  [saved results\q7_limitation.csv]

--- Objective check ----------------------------------------------------------------------------
  O1 same optimum on every instance              : MET
  O2 operations at n = 20: 20,971,520 -> 152 (99.999% fewer) : MET
  O3 40 packages in 0.16 ms (deadline 1000 ms)        : MET
  limitation: strongly correlated, n = 60: median 37,296 nodes, max 2,000,001, 1 of 5 hit the node cap
  [graph results\graphs\q7_comparison.png]
  [graph results\graphs\q7_limitation.png]
```

Figure 36: Nodes visited by V3 for uncorrelated and strongly correlated packages (median and maximum of 5 instances)



8.7 F. Defence of the Final Design

Was the prediction supported? Yes, for every version, and in one case more precisely than expected:

Table 8.8: Prediction Against Result at n = 20

Version	Predicted	Observed
V1	≈ 10× fewer operations; same 1,048,576 candidates	Exactly 10.0× (20,971,520 → 2,097,151); 1,048,576 candidates; time only 6.5× faster
V2	Candidates = feasible loads = 213,967	Exactly 213,967 — and equal to the feasible count from Q6 at every size (79, 281, 1,031, …)
V3	Far fewer candidates; weak for similar value per kg	1 candidate and 32 nodes; on strongly correlated data at n = 60 the median was 37,296 nodes (231× the uncorrelated median of 161), and 1 of 5 instances hit the cap

One part of the result was not predicted: V1's time improved only 6.5× while its operations improved 10×. The reason is that each node is a recursive Python function call, which costs more than one step of the original's inner loop. Counting operations measured the algorithmic saving correctly, but the cost per operation also changed.

Magnitude of improvement. All three objectives were met (Figure 35). O1: every instance returned the same optimum as the original and as dynamic programming. O2: at n = 20 the operations fell from 20,971,520 to 152, **99.999% fewer** (137,971×), and the time from 3,057 ms to 0.031 ms. O3: 40 packages were planned in 0.16 ms; even 80 packages took 0.33 ms. The deployment boundary moved from 17 packages to at least 80 for manifests like this one.

New costs and trade-offs. (1) The bound is now the main cost: at n = 20, 120 of the 152 operations were bound steps. (2) Preprocessing (one sort) and O(n) extra memory are needed. (3) The running time is no longer predictable from n alone — it ranged from 25 nodes to more than 2,000,000 depending on the data. (4) The worst case is still exponential: one strongly correlated instance with 60 packages was stopped at the 2,000,000-node cap without a proven optimum, and the slowest run in that group took 1.08 s. (5) The design returns one optimal load, not a list of alternatives.

When the original may still be preferable:

- **Very small problems.** For 8–10 packages every version finished in under 2 ms, so the original's simplicity (easy to verify, no recursion, no sorting) is worth more than the speed-up.
- **When every feasible load is wanted,** for example to show the planner the ten best alternatives or to explore trade-offs. Pruning deliberately skips those loads.
- **When the value of a load is not additive** — for instance, if medical kits are only useful together with water, or there are extra rules on volume or incompatible cargo. The

bound assumes additive values and one capacity; generate-and-test works with any evaluation function.

- **When a guaranteed, predictable running time matters more than the average:** the original always does exactly $n \cdot 2^n$ checks, whereas V3 can vary by a factor of thousands on unusual data.

Does the redesign sufficiently address the limitation? For the airlift problem as specified — packages with varied value per kg and one payload limit — yes: it removes all three diagnosed sources of waste, keeps the guarantee of the optimum, and turns a 17-package limit into an effectively unlimited one for realistic manifests. It does not change the fact that the problem can be hard: when all packages have almost the same value per kg, the bound cannot separate good from bad loads and the search grows quickly again. **My final engineering conclusion** is to deploy V3 with a node limit of about one second; if the limit is ever reached, the planner returns the best load found so far together with the bound, and reports how far from optimal it might be, instead of making the crew wait.

9. CONCLUSION

Across the seven questions the same lesson appeared in different forms: **the cost of a brute-force algorithm is decided by how much of the problem it refuses to rule out**. Insertion sort and the redesigned planner succeed because they use what they already know (existing order, partial sums, the payload limit, the best load so far). Selection sort, closest pair and the original planner are predictable but blind: they always do all of their work.

Table 9.1: Summary of Evidence and Decisions

Q	Key evidence (operation counts)	Decision
Q1	On 95%-sorted data insertion sort needed 25,461 comparisons; selection 499,500 and bubble 496,097. Bubble sort used all 999 passes on the turtle data.	No algorithm is always best. Recommend insertion sort for mostly-sorted orders; selection sort only if moves are very expensive.
Q2	Successful search = position; unsuccessful = n. Limit about 12,500 records at human request rates; 20 facilities: worst lookup 0.477 ms, margin ×2.1.	Keep sequential search for per-hangar lists; use a hash index for the combined 20-facility list.
Q3	Same n and m: 1.001 to 14.0 comparisons per window. The frequency model missed the NOP sleds by 2.8×.	Input structure, not size, decides the work; judge matchers on adversarial inputs too.
Q4	Pairs = $n(n-1)/2$ exactly; 99.9% of pairs were far apart in x; 2,000 drones took 223 ms.	Brute force is impractical beyond about 1,000–1,400 drones per 100 ms check.
Q5	Orientation tests = $n(n-1)(n-2)/2$ exactly (16.2 million at n = 320); the naive rule accepted 24 edges instead of 5.	Use the one-side rule plus the collinear "between" rule with exact integer arithmetic.
Q6	Candidates = 2^n exactly; item checks ×4.44–×5.00 per two packages; n = 18 measured at 1.284 s.	Deploy exhaustive search for at most 17 packages within 1 s.
Q7	Same optimum; 20,971,520 → 152 operations at n = 20; 80 packages in 0.33 ms; strongly correlated data still hard.	Deploy the redesign (V3) with a node limit and a reported optimality gap.

10. REFERENCES

1. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012 — Ch. 3 (brute force and exhaustive search) and Ch. 12 (branch-and-bound).
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
3. D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998 — inversions and insertion sort.
4. A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979 — used only as the independent check in Q5.
5. D. Pisinger, "Where are the hard knapsack problems?" *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, 2005 — strongly correlated knapsack instances.
6. J. Nielsen, *Usability Engineering*. Academic Press, 1993 — response-time limits of 0.1 s and 1 s used in Q2 and Q6.

11. APPENDIX — REPRODUCIBILITY AND SUBMISSION

11.1 How to Reproduce the Results

1. Install Python 3 and the graph library: `python -m pip install matplotlib`.
2. Set `REG_NO = 192511209` and `NAME = "Jason Joshva M J"` in `config.py`.
3. Run each question (`python q1_sorting.py` … `python q7_redesign.py`) or all of them with `python run_all.py`.
4. The operation counts are identical on every run with the same registration number; the times vary with the machine and its load.

11.2 Submitted Files

Table 11.1: Repository Contents

File or folder	Contents
config.py, common.py	Registration number and name; shared helpers (seed, tables, CSV output, graphs)
q1_sorting.py … q7_redesign.py	One program per question; run_all.py runs all seven
results/*.csv	Raw experimental data for every table in this report
results/q1_console.txt … q7_console.txt	Complete terminal output of each run
results/data/	The input datasets: order scores, hangar records, logs and packet dump, drone positions, station sets, package manifest
results/graphs/	The 15 graphs used in this report

11.3 Submission Checklist

- Report (this document) with the evidence for Q1–Q7.
- Source code for all seven solutions.
- Raw experimental data (results/ and results/data/).
- Tables and graphs (in this report and results/graphs/).
- Fill in the programme and the repository link in Table 1.1 before submitting.